

Diagnostic de l'agent SAR (champion 0.9151)

Notebook de diagnostic pour `cf_search_and_rescue` . Il lit les sorties de `analysis/collect_rollouts.py` (rollouts instrumentés en process) et, en option, le JSON d'un `swarm benchmark --summary-json-out` et un second run (variante) pour un A/B apparié.

Produire les données (sur le VPS Linux, depuis la racine du repo) :

```
# champion, 1 100 seeds (plusieurs heures) – ou --per-type 20 pour itérer
python analysis/collect_rollouts.py --model TASK/champion/submission.zip \
  --seed-file TASK/practice_seeds.json --out analysis/data/champion --workers 4
# variante (ex. désactiver la branche RGB du champion)
python analysis/collect_rollouts.py --model TASK/champion/submission.zip \
  --seed-file TASK/practice_seeds.json --out analysis/data/no_rgb --agent-env
KT_RP=0
```

Plan

\$	Question
1	Vue d'ensemble : score, succès, où partent les points
2	Échecs : taxonomie, quand, où
3	Terme temps : phases de la mission, retard vs temps cible
4	Terme sécurité : clearance
5	Facteurs géométriques : qu'est-ce qui prédit l'échec
6	Perception & couverture de recherche
7	Phase finale : approche et hover de confirmation
8	Internes de l'agent : classifieur de type, routes, RGB, escape, évitement
9	Cinématique
10	Budget de calcul
11	Galerie de trajectoires
12	Cohérence avec le benchmark Docker
13	A/B apparié champion vs variante
14	Puissance statistique
15	Synthèse : priorisation des gains

Debrief : run complet 1 100 seeds (VPS, 1–2 oct. 2026)

Deux runs sur les mêmes 1 100 seeds de `practice_seeds.json` (184 environ par type, aucun `office`) : les **rollouts instrumentés** en process (`collect_rollouts.py` , 12 h) et le **benchmark officiel** Docker (`swarm benchmark` , 5 h 18). Aucune erreur d'exécution des deux côtés (1 100/1 100 `seed_done`).

	Rollouts (process)	Benchmark Docker
Score moyen	0.8900 (IC 95 % [0.872, 0.907])	0.8917
Succès	991/1 100 (90.1 %)	993/1 100 (90.3 %)
Désaccords succès/échec seed à seed	6 / 1 100 (toutes en <code>city</code>)	6 / 1 100

Ce qu'il faut retenir

1. **Le score des practice seeds est de 0.892, pas 0.9151.** L'IC à 95 % exclut 0.9151 : le score publié vient d'un autre jeu de seeds (ou d'un autre mélange de types, **office** compris). Il faut le vérifier avant de comparer une variante au chiffre publié. Les comparaisons doivent se faire en A/B apparié sur ces seeds.
2. **89 % de la perte vient des échecs** (0.098 pt). Les termes temps (0.0065) et sécurité (0.0055) pèsent peu : un succès vaut déjà 0.987 en moyenne.
3. **La perte est concentrée sur quatre types.** Mountain 0.780 (un tiers de la perte), forest 0.835, warehouse 0.879, village 0.897. City 0.946 et open 1.000 sont quasi résolus.
4. **Le premier mode d'échec est perceptif, pas le contrôle** : dans 44 échecs (40 %), la victime a été visible mais n'a jamais été approchée. Elle n'apparaît que petite (21 px au max contre 47 pour un succès), de loin (> 11 m), pendant environ 5 s, puis l'agent passe à côté. S'y ajoutent 29 échecs où elle n'a jamais été visible (recherche, mountain surtout).
5. **Les victimes allongées sont un sous-cas faible** : 70 % de succès contre 92 % pour les autres ($n = 77$). En mountain on tombe à 61 % (44 seeds), en forest à 55 %. Les autres facteurs géométriques (distances, dénivelé) n'ont presque pas d'effet.
6. **Deux défauts nets et peu coûteux à corriger** :
 - 7 crashes warehouse **au décollage** ($t = 0.32$ s). Le drone monte à la verticale dans un obstacle situé à environ 0.5 m au-dessus. Gain : +0.006 sur 7 seeds identifiées.
 - 10 échecs « < 2 m sans prédicat » : le drone reste **trop haut** (> 4 m au-dessus de la victime dans 95 % des pas). Gain : +0.009.
7. **Les rollouts en process sont un proxy fidèle du benchmark** (écart de 0.0017, 6 désaccords en **city** seulement, où le comportement n'est pas déterministe). On peut itérer en process et ne valider en Docker qu'à la fin.
8. **Le budget de calcul ne pose pas problème** : 0 strike fatal en Docker, environ 95 ms par pas en moyenne et 250 ms au p95.

Plan proposé, par rapport gain/effort, chaque étape validée par un A/B apparié (section 13) : (a) contrôle du décollage en warehouse, (b) descente forcée quand la victime est à < 2 m horizontalement mais à plus de 4 m de hauteur, (c) mémorisation des détections fugaces pour engager l'approche (cible du plus gros gisement, +0.039), (d) recherche en mountain (RGB : 90 % de succès avec verrou contre 77 % sans, verrou obtenu dans seulement 21 % des épisodes), (e) crashes forest. Pour mesurer +0.005, il faut environ 6 seeds réussies de plus en net : c'est hors de portée d'une comparaison non appariée (± 0.018), mais facile à voir en apparié.

0. Configuration

Chargement et contrôles de cohérence

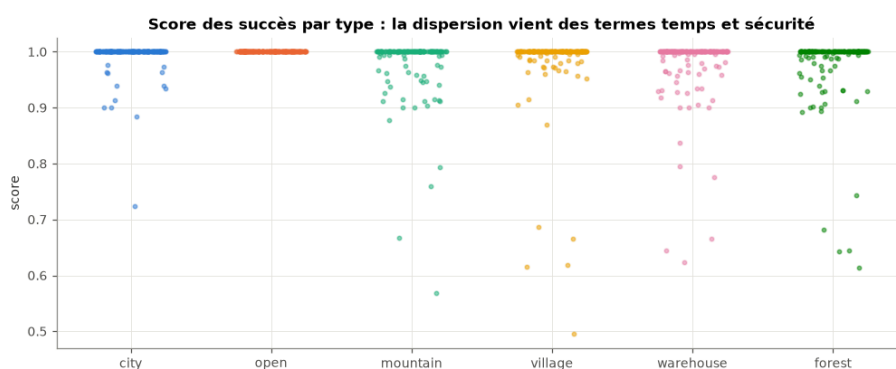
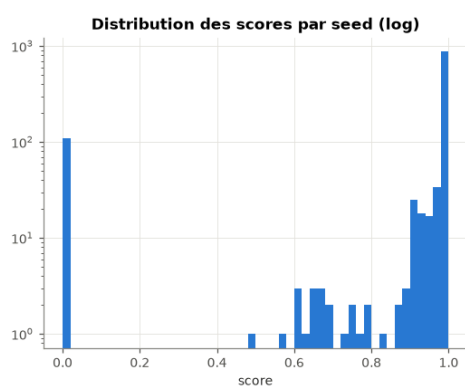
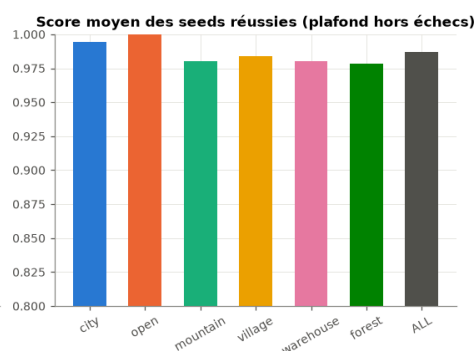
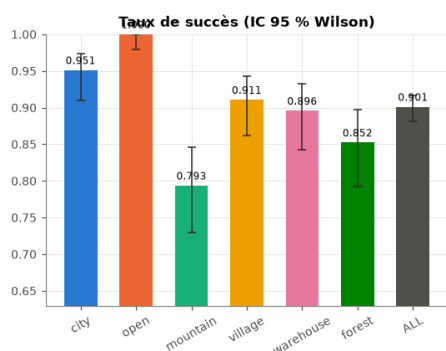
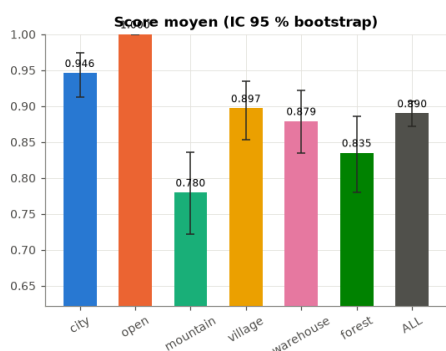
1100 épisodes dans D:\projets IA\SWARM\sar-task\analysis\data\champion_full
trajectoires disponibles : 1100/1100

	n	succes	score	act_errors	collision_sur_succes
env_type					
city	184.0	0.951	0.946	0.0	0.0
open	184.0	1.000	1.000	0.0	0.0
mountain	184.0	0.793	0.780	0.0	0.0
village	190.0	0.911	0.897	0.0	0.0
warehouse	182.0	0.896	0.879	0.0	0.0
forest	176.0	0.852	0.835	0.0	0.0
TOTAL	1100.0	0.901	0.890	0.0	0.0

1. Vue d'ensemble

Le score par seed est quasi binaire (≈ 0.95 ou 0.01). Les IC sont donc larges sur de petits échantillons : un IC à 95 % par bootstrap (score) et de Wilson (taux de succès) est affiché partout.

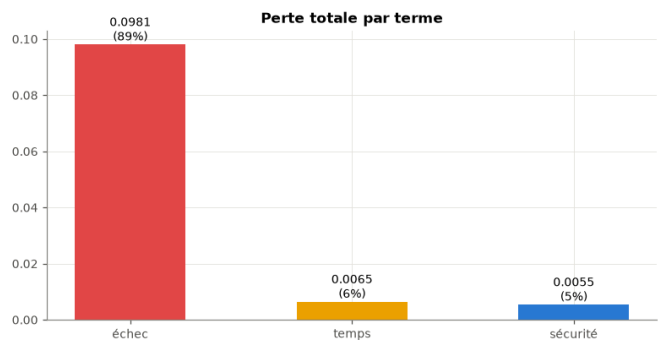
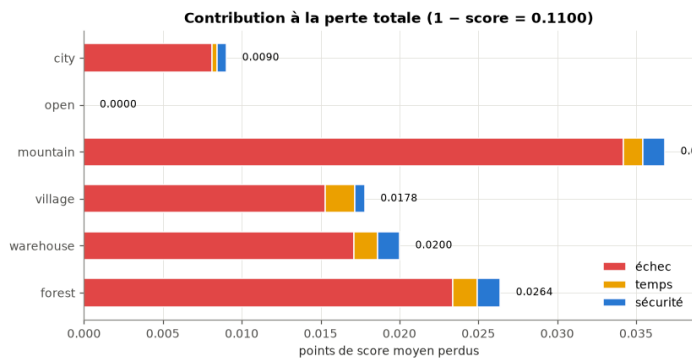
	n	score	s_lo	s_hi	succ	p_lo	p_hi	score_si_succes
type								
city	184	0.946	0.913	0.975	0.951	0.910	0.974	0.994
open	184	1.000	1.000	1.000	1.000	0.980	1.000	1.000
mountain	184	0.780	0.722	0.836	0.793	0.729	0.846	0.980
village	190	0.897	0.853	0.935	0.911	0.861	0.943	0.984
warehouse	182	0.879	0.835	0.922	0.896	0.843	0.932	0.980
forest	176	0.835	0.780	0.886	0.852	0.792	0.897	0.978
ALL	1100	0.890	0.872	0.907	0.901	0.882	0.917	0.987



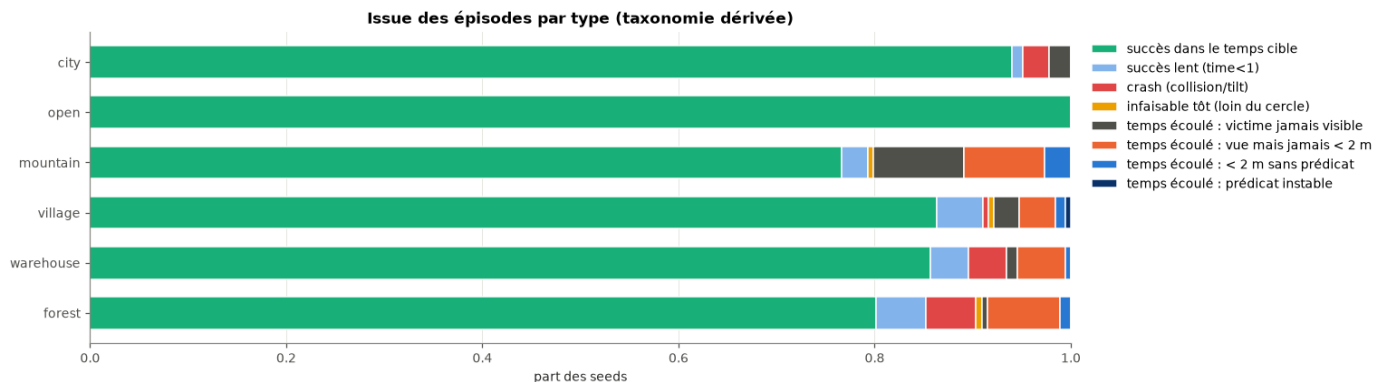
1.1 Où partent les points ?

Perte par seed = 1 – score, décomposée en **échec** (0.99 par seed ratée, ou 0.99 si un succès est annulé par une collision), **temps** (0.45 × (1 – time_term)) et **sécurité** (0.10 × (1 – safety)). La contribution d'un type à la perte totale est pondérée par sa part de seeds : les barres somment à 1 – score moyen global.

	loss_fail (pts %)	loss_time (pts %)	loss_safety (pts %)	total (pts %)
env_type				
city	0.81	0.04	0.06	0.90
open	0.00	0.00	0.00	0.00
mountain	3.42	0.12	0.14	3.68
village	1.53	0.19	0.06	1.78
warehouse	1.71	0.15	0.14	2.00
forest	2.34	0.15	0.14	2.64



outcome	success_fast	success_slow	crash	infeasible	to_never_seen	to_seen_not_close	to_close_no_pred	to_pred_unstable	All
env_type									
city	173	2	5	0	4	0	0	0	184
open	184	0	0	0	0	0	0	0	184
mountain	141	5	0	1	17	15	5	0	184
village	164	9	1	1	5	7	2	1	190
warehouse	156	7	7	0	2	9	1	0	182
forest	141	9	9	1	1	13	2	0	176
All	959	32	22	3	29	44	10	1	1100



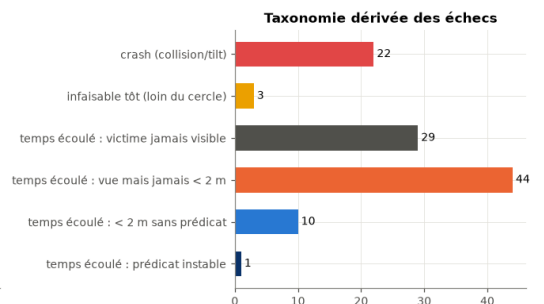
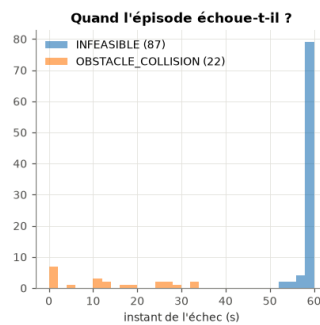
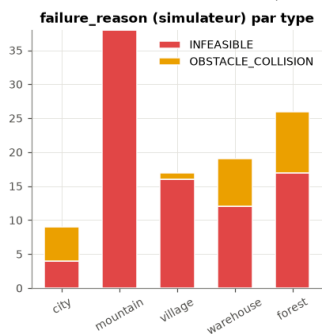
Lecture (1 100 seeds). Open est parfait (184/184). La perte totale (11 pts) se répartit en mountain 3.7, forest 2.6, warehouse 2.0, village 1.8 et city 0.9. Presque tout vient des échecs (rouge). Les succès lents (success_slow , 32 seeds) et la sécurité ne coûtent qu'environ 0.6 pt chacun.

2. Échecs

Taxonomie : `failure_reason` du simulateur, raffinée avec la trajectoire pour les épisodes à court de temps (victime jamais visible → problème de **recherche** ; vue mais jamais approchée → problème de **détection/décision** ; approchée sans prédicat → problème de **hover** ; prédicat instable → problème de **contrôle fin**).

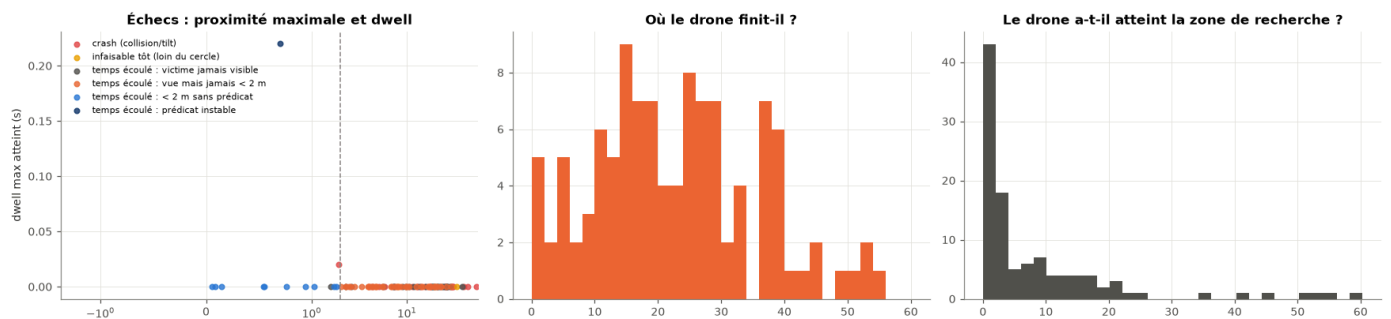
NB : le simulateur ne renvoie presque jamais `TIMEOUT` : `infeasible()` tronque l'épisode dès que le temps restant ne couvre plus les 2 s de hover, soit $t \approx 58$ s pour un drone dans le cercle. Ces `INFEASIBLE` tardifs sont donc traités comme des timeouts ; seuls ceux tronqués tôt (drone loin du cercle) restent « infaisable tôt ».

109 échecs / 1100 (9.9%)



	group	seed	failure_reason	outcome	sim_time	d_start_centre	d_centre_victim	victim_dz_start	first_visible_t
20	type1_city	1000027	OBSTACLE_COLLISION	crash	10.96	85.561	20.460	-0.070	NaN
43	type1_city	1000054	OBSTACLE_COLLISION	crash	11.02	24.102	21.825	0.230	11.0
790	type1_city	1000729	OBSTACLE_COLLISION	crash	27.48	58.164	29.232	-2.309	NaN
938	type1_city	1000909	OBSTACLE_COLLISION	crash	19.90	48.962	14.366	-3.151	NaN
1067	type1_city	1001058	OBSTACLE_COLLISION	crash	32.48	59.323	29.283	-2.797	26.7
256	type4_village	1000304	OBSTACLE_COLLISION	crash	5.08	8.448	10.866	0.728	NaN
71	type5_warehouse	1000059	OBSTACLE_COLLISION	crash	0.32	51.747	21.933	1.634	NaN
209	type5_warehouse	1000193	OBSTACLE_COLLISION	crash	0.32	14.987	28.596	1.634	NaN
407	type5_warehouse	1000397	OBSTACLE_COLLISION	crash	0.32	54.429	27.197	1.634	NaN
455	type5_warehouse	1000448	OBSTACLE_COLLISION	crash	0.36	11.198	1.066	1.634	NaN
887	type5_warehouse	1000875	OBSTACLE_COLLISION	crash	0.36	52.916	25.857	1.634	NaN
905	type5_warehouse	1000900	OBSTACLE_COLLISION	crash	0.32	25.656	5.071	1.634	NaN
947	type5_warehouse	1000950	OBSTACLE_COLLISION	crash	0.32	35.664	28.293	1.634	NaN
202	type6_forest	1000190	OBSTACLE_COLLISION	crash	25.04	20.565	15.314	-0.167	1.0
240	type6_forest	1000219	OBSTACLE_COLLISION	crash	16.38	34.560	22.271	0.408	0.1
322	type6_forest	1000314	OBSTACLE_COLLISION	crash	26.62	32.315	27.042	-1.560	17.2
374	type6_forest	1000355	OBSTACLE_COLLISION	crash	33.90	2.132	28.561	-1.097	4.0
405	type6_forest	1000407	OBSTACLE_COLLISION	crash	12.50	17.103	21.413	-1.486	0.1
554	type6_forest	1000563	OBSTACLE_COLLISION	crash	11.14	58.427	11.289	1.012	NaN
668	type6_forest	1000668	OBSTACLE_COLLISION	crash	28.56	62.473	18.319	-0.887	18.7
741	type6_forest	1000747	OBSTACLE_COLLISION	crash	13.44	34.001	14.188	-2.056	11.6
888	type6_forest	1000879	OBSTACLE_COLLISION	crash	25.18	37.591	24.521	0.798	17.0
283	type3_mountain	1000255	INFEASIBLE	infeasible	54.98	61.996	23.784	-26.376	NaN
477	type4_village	1000545	INFEASIBLE	infeasible	52.90	31.290	29.519	0.516	NaN
99	type6_forest	1000085	INFEASIBLE	infeasible	53.16	56.716	11.303	-1.390	NaN
600	type1_city	1000528	INFEASIBLE	to_never_seen	58.00	43.033	29.878	-2.274	NaN
894	type1_city	1000854	INFEASIBLE	to_never_seen	58.00	67.895	29.398	-2.090	NaN
993	type1_city	1000966	INFEASIBLE	to_never_seen	58.00	76.503	26.512	-2.444	NaN
1055	type1_city	1001046	INFEASIBLE	to_never_seen	58.00	24.482	27.401	-0.895	NaN
50	type3_mountain	1000031	INFEASIBLE	to_never_seen	58.00	96.879	26.114	40.123	NaN
138	type3_mountain	1000119	INFEASIBLE	to_never_seen	58.00	74.784	16.358	3.592	NaN
158	type3_mountain	1000129	INFEASIBLE	to_never_seen	58.00	69.606	10.708	-12.644	NaN
212	type3_mountain	1000195	INFEASIBLE	to_never_seen	55.60	15.690	29.935	-6.559	NaN
286	type3_mountain	1000256	INFEASIBLE	to_never_seen	58.00	59.062	27.232	9.703	NaN
436	type3_mountain	1000380	INFEASIBLE	to_never_seen	58.00	51.748	29.198	-28.578	NaN
481	type3_mountain	1000424	INFEASIBLE	to_never_seen	58.00	103.925	17.236	3.134	NaN
558	type3_mountain	1000550	INFEASIBLE	to_never_seen	58.00	67.801	10.971	-21.115	NaN
654	type3_mountain	1000630	INFEASIBLE	to_never_seen	56.56	70.218	4.520	6.366	NaN
715	type3_mountain	1000724	INFEASIBLE	to_never_seen	58.00	67.776	24.204	-11.744	NaN
857	type3_mountain	1000852	INFEASIBLE	to_never_seen	58.00	70.226	29.545	-34.548	NaN
861	type3_mountain	1000860	INFEASIBLE	to_never_seen	58.00	106.668	21.386	-16.146	NaN
873	type3_mountain	1000868	INFEASIBLE	to_never_seen	58.00	120.786	16.475	-16.209	NaN
899	type3_mountain	1000881	INFEASIBLE	to_never_seen	58.00	53.531	24.462	-22.848	NaN
924	type3_mountain	1000892	INFEASIBLE	to_never_seen	58.00	114.778	7.784	-7.470	NaN
1040	type3_mountain	1001010	INFEASIBLE	to_never_seen	58.00	109.006	10.907	-17.949	NaN
1077	type3_mountain	1001036	INFEASIBLE	to_never_seen	58.00	97.477	4.783	-14.671	NaN

	group	seed	failure_reason	outcome	sim_time	d_start_centre	d_centre_victim	victim_dz_start	first_visible_t
23	type4_village	1000029	INFEASIBLE	to_never_seen	58.00	21.758	19.769	-3.647	NaN
387	type4_village	1000435	INFEASIBLE	to_never_seen	58.00	51.947	29.532	-3.262	NaN
513	type4_village	1000565	INFEASIBLE	to_never_seen	58.00	25.096	24.763	0.478	NaN
844	type4_village	1000914	INFEASIBLE	to_never_seen	57.40	49.136	26.048	0.754	NaN
866	type4_village	1000944	INFEASIBLE	to_never_seen	58.00	8.883	11.442	-4.547	NaN
511	type5_warehouse	1000493	INFEASIBLE	to_never_seen	58.00	25.341	21.937	1.634	NaN
818	type5_warehouse	1000784	INFEASIBLE	to_never_seen	58.00	28.819	22.864	1.724	NaN
602	type6_forest	1000587	INFEASIBLE	to_never_seen	58.00	35.888	21.499	-0.381	NaN
97	type3_mountain	1000083	INFEASIBLE	to_seen_not_close	58.00	117.502	29.133	28.506	44.3
244	type3_mountain	1000221	INFEASIBLE	to_seen_not_close	58.00	18.868	25.167	-12.175	36.6
246	type3_mountain	1000226	INFEASIBLE	to_seen_not_close	58.00	55.656	13.425	-22.811	24.6
350	type3_mountain	1000303	INFEASIBLE	to_seen_not_close	58.00	29.496	21.518	33.354	12.9
354	type3_mountain	1000310	INFEASIBLE	to_seen_not_close	58.00	80.735	22.725	3.073	31.9
417	type3_mountain	1000363	INFEASIBLE	to_seen_not_close	58.00	79.160	18.402	-14.913	36.3



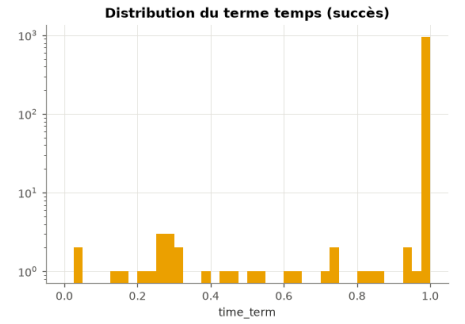
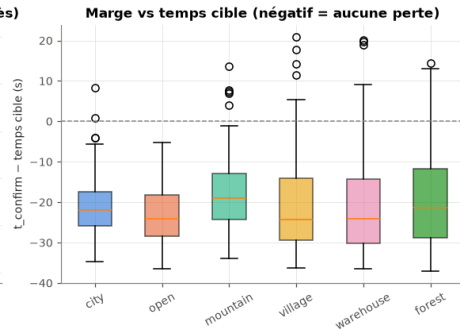
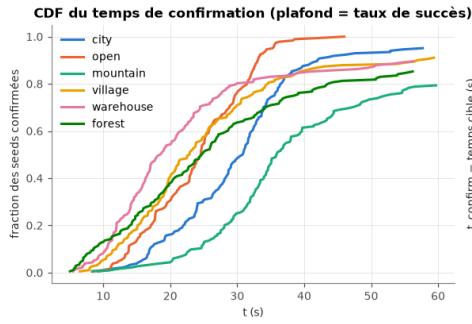
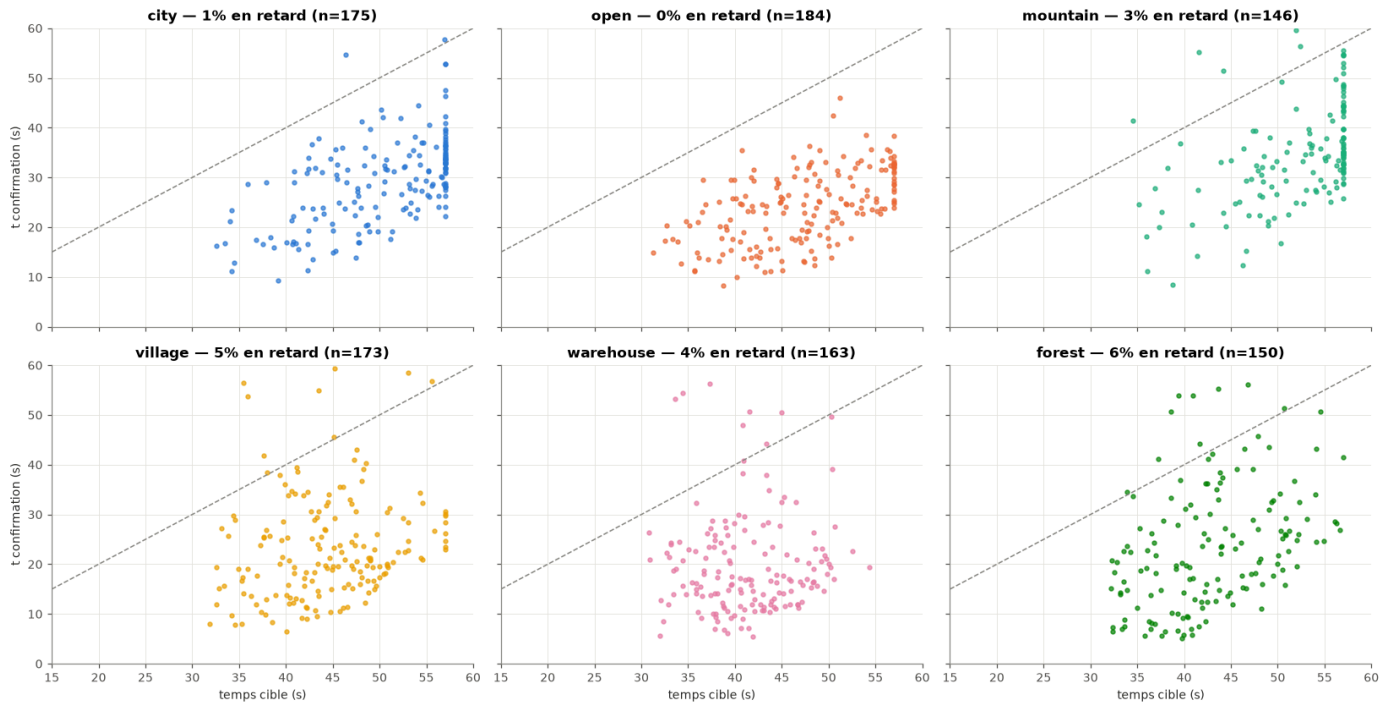
Lecture. 109 échecs. Les timeouts dominent : vue sans approche 44, jamais vue 29, < 2 m sans prédicat 10, prédicat instable 1. Viennent ensuite 22 crashes et 3 infaisables tôt.

- Les **crashes warehouse (7)** surviennent tous à **t = 0.32–0.36 s**, au décollage (voir 16.2). Les crashes forest (9) et city (5) ont lieu en vol, entre 11 et 34 s.
- Pour les timeouts « jamais vue », le drone a bien atteint le centre (distance min. médiane 1.3 m) : le problème est le **balayage**, pas la navigation.

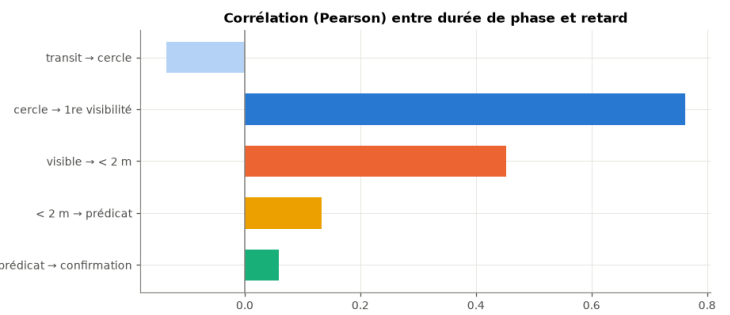
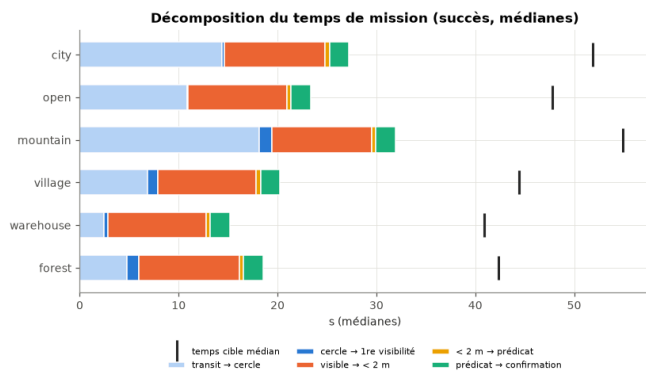
3. Terme temps

$\text{target_time} = 1.03 \times (\text{d}(\text{start}, \text{centre})/3 + 27.5 + 2)$. Toute seconde au-delà coûte 0.45 / (60 – target) points. On décompose chaque succès en phases : transit → entrée dans le cercle → première visibilité géométrique → < 2 m → prédicat → confirmation.

Temps de confirmation vs temps cible (au-dessus de la diagonale = perte de points)



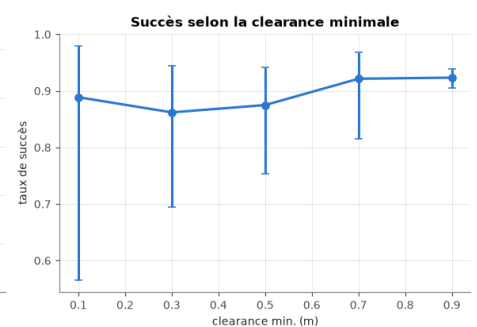
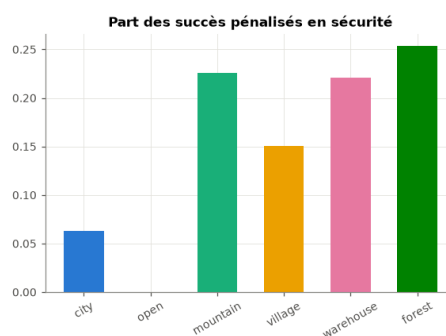
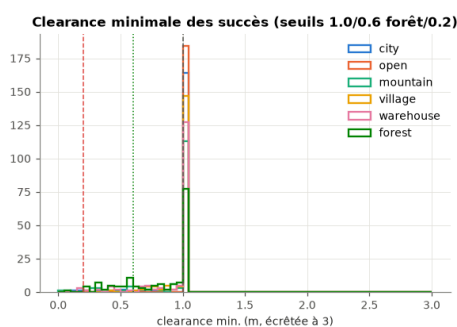
	env_type	city	open	mountain	village	warehouse	forest
transit → cercle	count	174.0	184.0	146.0	173.0	163.0	149.0
	mean	14.6	11.1	18.3	7.5	4.4	7.2
	std	8.8	8.1	9.8	6.4	5.1	8.2
	min	0.0	0.0	0.0	0.0	0.0	0.0
	25%	8.1	4.6	11.9	0.0	0.0	0.0
	50%	14.4	10.8	18.1	6.8	2.5	4.8
	75%	20.5	16.6	24.8	11.6	7.9	12.1
	max	34.1	31.8	46.3	24.2	19.6	37.3
cercle → 1re visibilité	count	175.0	184.0	145.0	173.0	163.0	150.0
	mean	1.7	0.6	1.8	3.4	1.3	3.4
	std	7.9	7.4	7.1	10.2	7.5	10.1
	min	-12.2	-12.3	-10.4	-12.7	-12.1	-14.3
	25%	-4.1	-5.7	-3.2	-3.8	-5.0	-3.4
	50%	0.2	0.1	1.3	1.1	0.4	1.3
	75%	6.9	6.7	5.7	9.0	6.1	7.9
	max	25.8	20.7	23.4	41.2	24.7	38.1
visible → < 2 m	count	175.0	184.0	145.0	173.0	163.0	150.0
	mean	10.6	9.9	11.9	9.7	11.4	9.9
	std	2.4	1.7	6.3	5.3	8.0	7.2
	min	2.7	-8.7	1.4	-12.8	-10.6	-34.6
	25%	10.0	9.9	9.8	7.4	8.1	7.0
	50%	10.2	10.1	10.1	9.9	9.9	10.1
	75%	11.0	10.4	11.9	10.2	11.0	11.5
	max	32.7	12.4	45.9	43.9	52.0	40.3
< 2 m → prédicat	count	175.0	184.0	146.0	173.0	163.0	150.0
	mean	0.5	0.6	0.4	0.8	0.7	1.1
	std	0.2	1.3	0.3	2.0	2.0	4.5
	min	0.2	0.2	0.0	0.1	0.2	0.1
	25%	0.4	0.4	0.2	0.4	0.3	0.4
	50%	0.4	0.4	0.4	0.4	0.4	0.5
	75%	0.5	0.4	0.6	0.5	0.5	0.6
	max	2.6	16.1	2.3	23.7	20.6	51.8
prédicat → confirmation	count	175.0	184.0	146.0	173.0	163.0	150.0
	mean	2.0	2.0	2.0	2.0	2.0	2.0
	std	0.0	0.0	0.6	0.0	0.0	0.0
	min	2.0	2.0	2.0	2.0	2.0	2.0
	25%	2.0	2.0	2.0	2.0	2.0	2.0
	50%	2.0	2.0	2.0	2.0	2.0	2.0
	75%	2.0	2.0	2.0	2.0	2.0	2.0
	max	2.0	2.0	9.3	2.0	2.0	2.0



Lecture. Pour les succès, la confirmation suit presque toujours le même déroulé : prédicat obtenu 0.4 s après le passage sous 2 m, puis exactement 2.0 s de hover. La phase « visible → < 2 m » dure environ **10 s dans tous les types** : la victime devient visible à 24–29 m et le drone s'approche à 2–2.7 m/s. Cette phase est donc limitée par la vitesse de déplacement. Le terme temps ne coûte que 0.65 pt : le gain possible y est faible.

4. Terme sécurité (clearance minimale)

env_type	perte_moy	frac_pénalisé
city	0.004	0.063
open	0.000	0.000
mountain	0.011	0.226
village	0.004	0.150
warehouse	0.009	0.221
forest	0.011	0.253

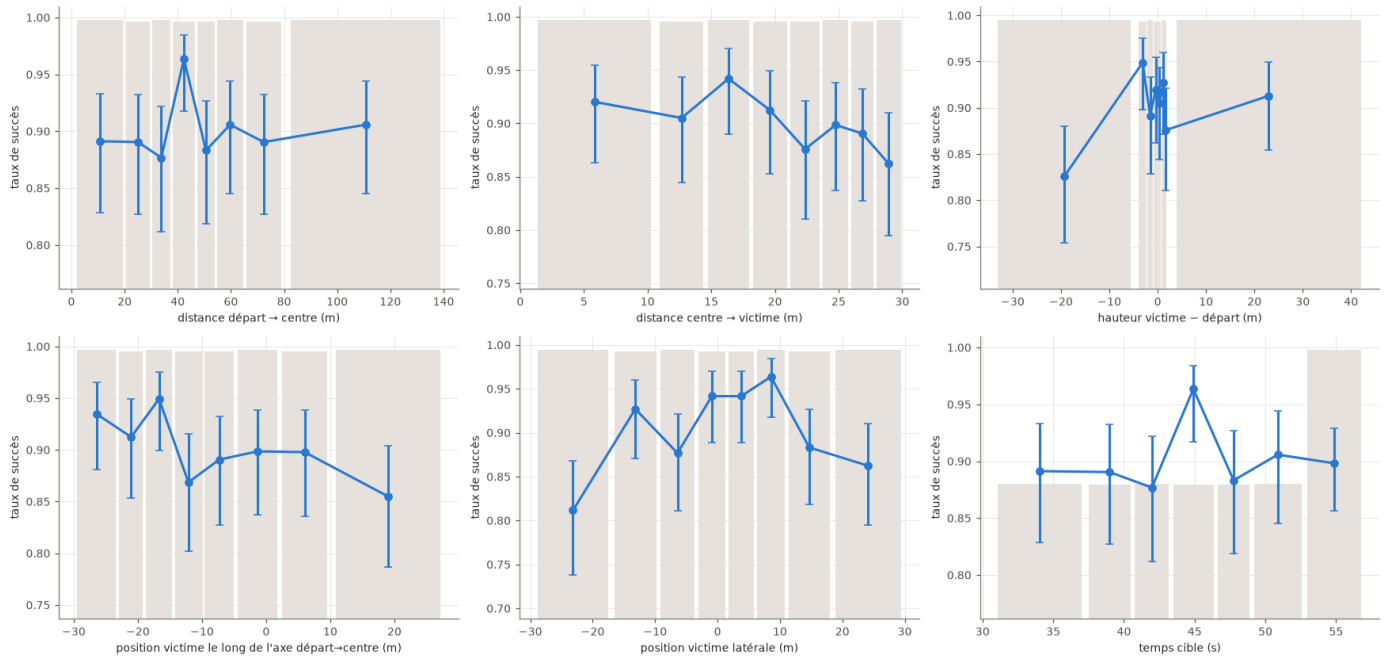


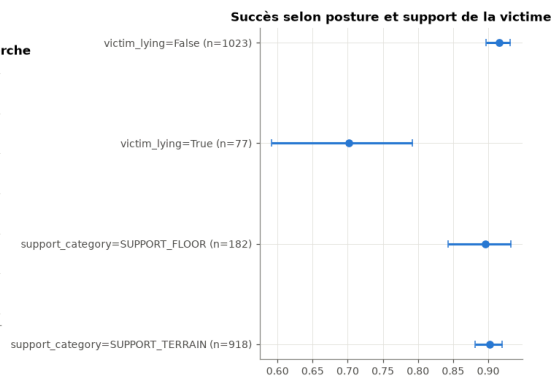
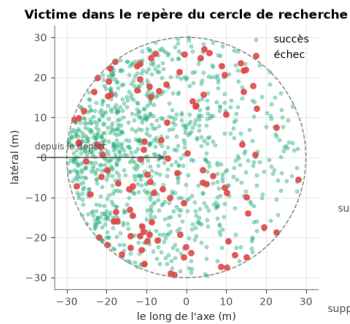
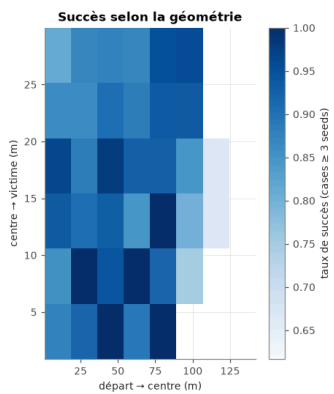
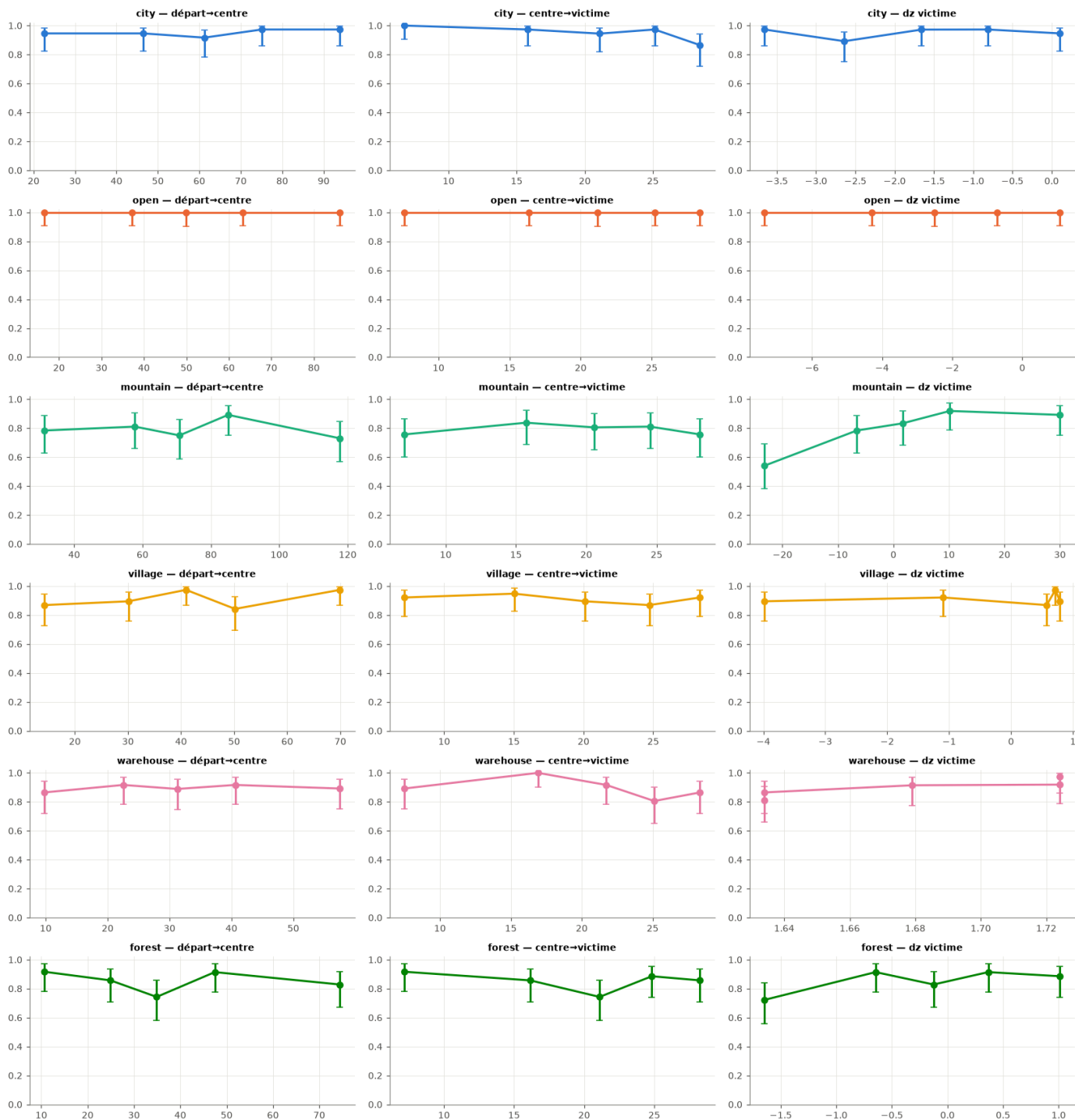
Lecture. La sécurité coûte 0.55 pt. Environ 22–25 % des succès sont pénalisés en mountain, warehouse et forest, 6 % en city et aucun en open. La pénalité par seed est petite (≈ 0.01) : ce n'est pas une priorité.

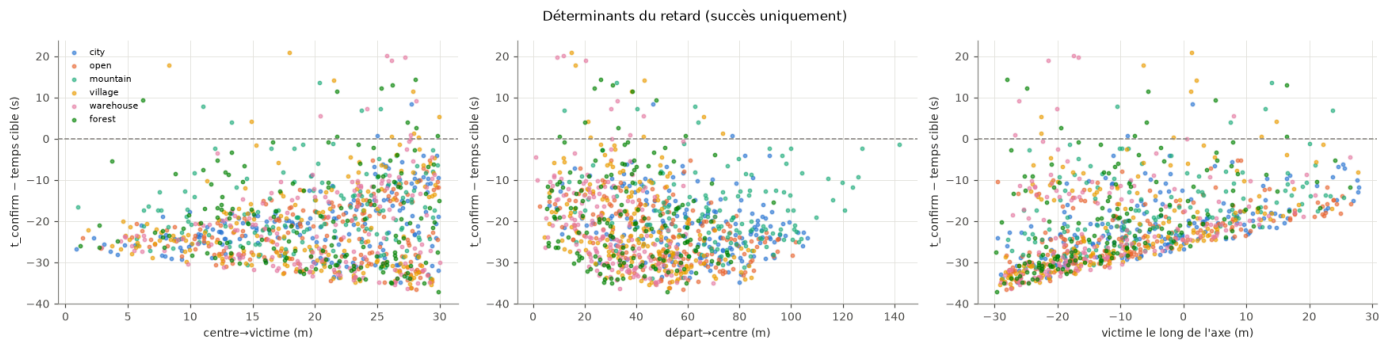
5. Facteurs géométriques

Taux de succès (IC Wilson) en fonction des paramètres de la tâche, tous connus avant l'épisode. Un facteur fortement prédictif indique un sous-cas que l'agent gère mal.

Taux de succès par quantile de facteur (barres grises = effectif)







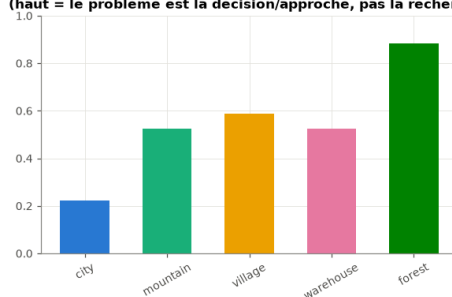
Lecture. Les facteurs connus avant l'épisode prédisent mal l'échec. Par quartile, le taux de succès reste entre 88 et 93 % pour les distances départ→centre et centre→victime et pour le dénivelé. Il baisse légèrement quand la victime est en bord de cercle (> 21 m du centre : 88 % contre 92 %). Le seul facteur fort est la **posture** : une victime allongée réussit à 70 % contre 92 % (voir 16.1).

6. Perception et couverture de la recherche

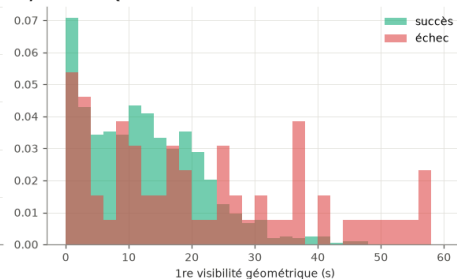
vis = vérité terrain géométrique (victime dans le champ, < 30 m, non occultée), calculée tous les 5 pas. La **couverture** est un proxy : fraction du disque de recherche passée à moins de 10 m du drone.

vue	False		True	
	succès	False	True	False
env_type				
city	7	0	2	175
open	0	0	0	184
mountain	18	1	20	145
village	7	0	10	173
warehouse	9	0	10	163
forest	3	0	23	150

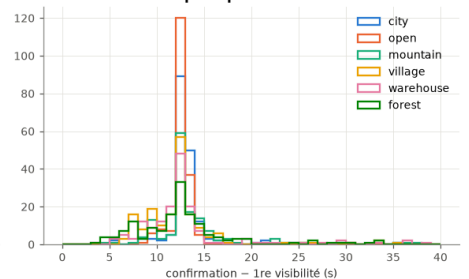
Échecs : part où la victime a été VISIBLE
(haut = le problème est la décision/approche, pas la recherche)

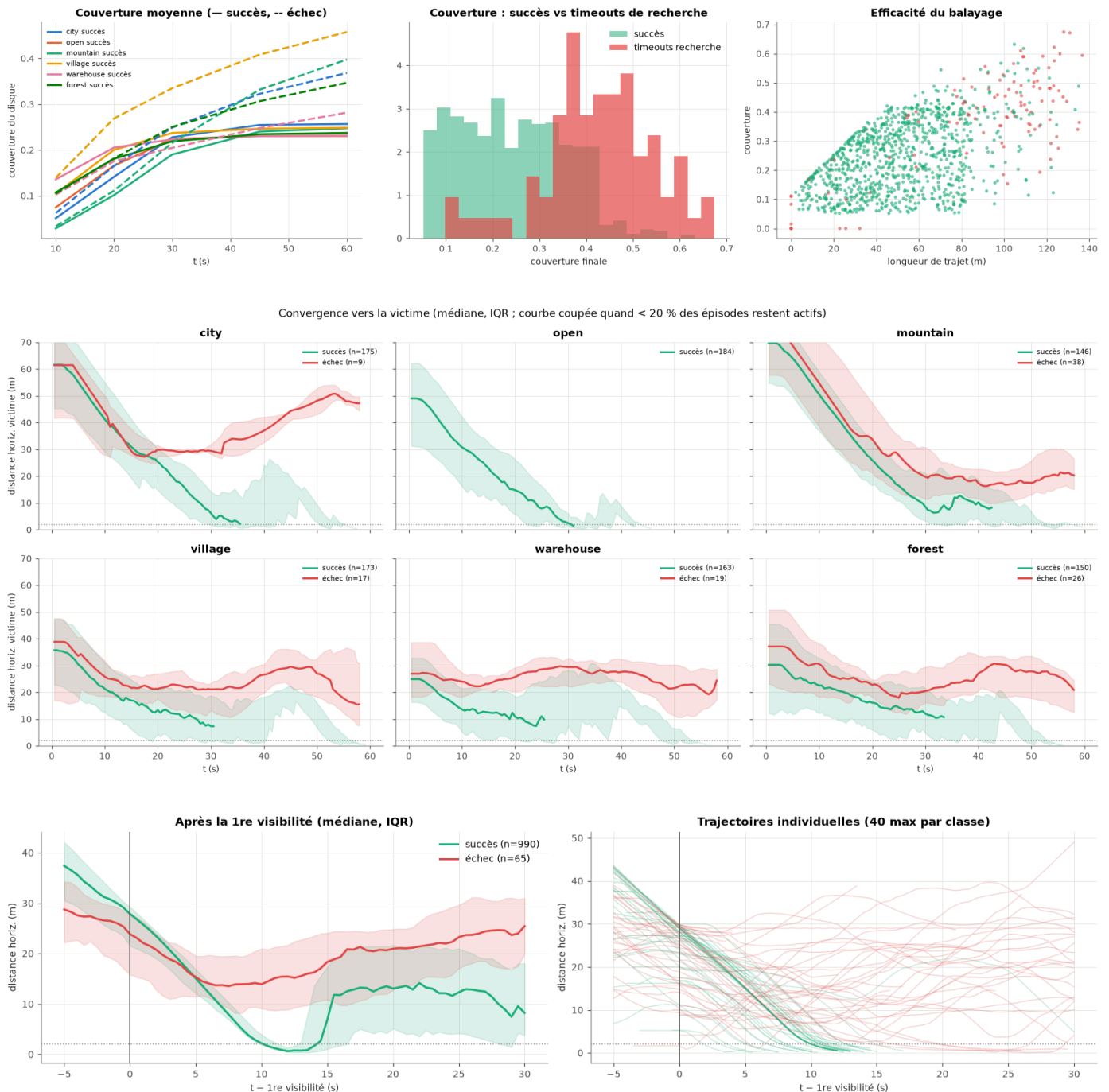


Quand la victime devient-elle visible ?



Latence perception → confirmation

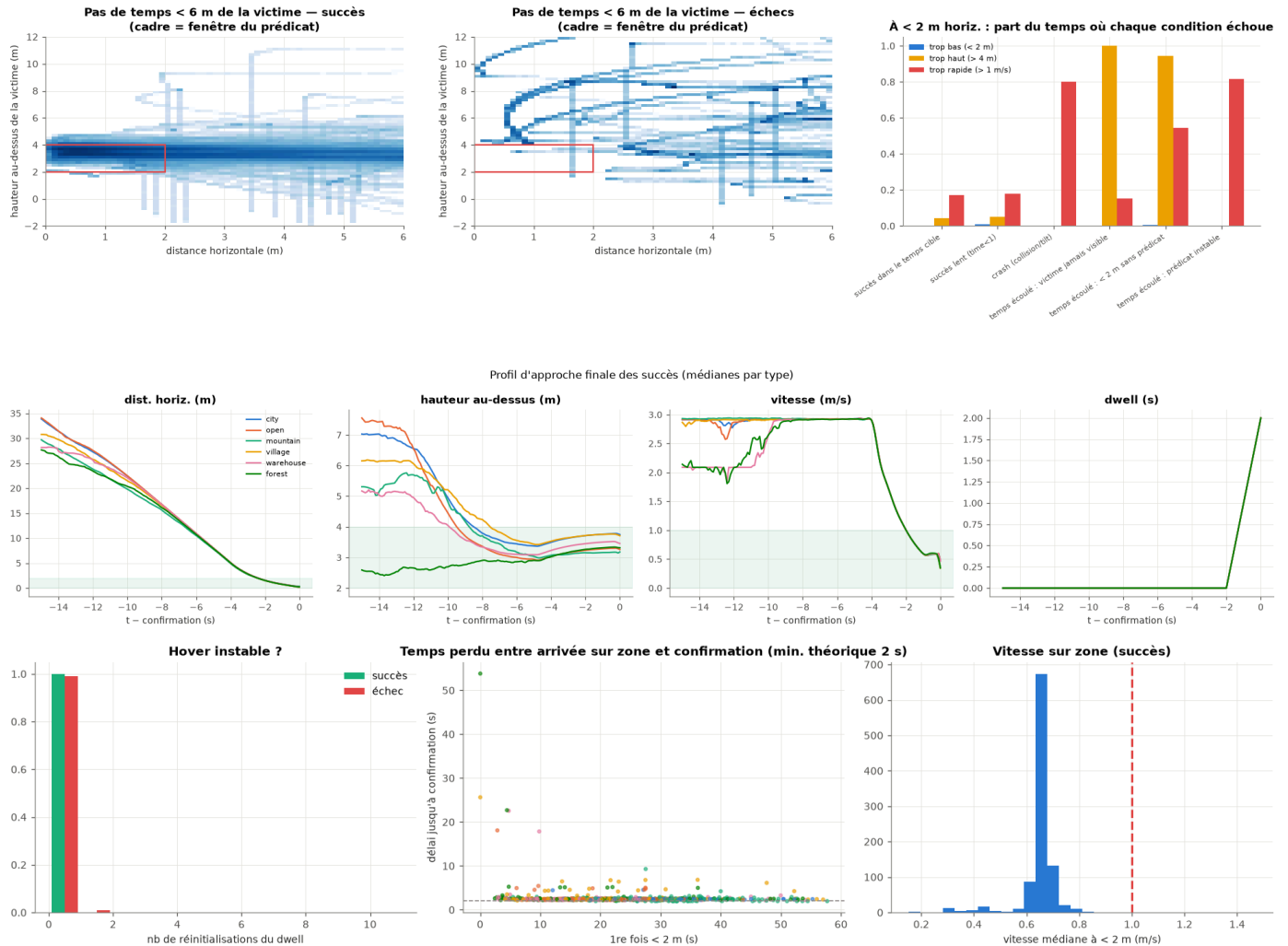




Lecture. Le « vue mais jamais approchée » est le gisement principal (44 seeds). La victime est visible au total 4.7 s seulement (médiane), avec 21 px au maximum (contre 47 pour les succès), et le drone ne passe jamais à moins de 11.7 m d'elle pendant qu'elle est visible. Elle sort du champ avant que la politique ne s'engage. Les épisodes en échec couvrent davantage le disque (0.39 contre 0.24) : le drone cherche longtemps et a déjà « vu » la cible. Il faudrait **mémoriser la dernière détection** pour y revenir.

7. Phase finale : approche et hover de confirmation

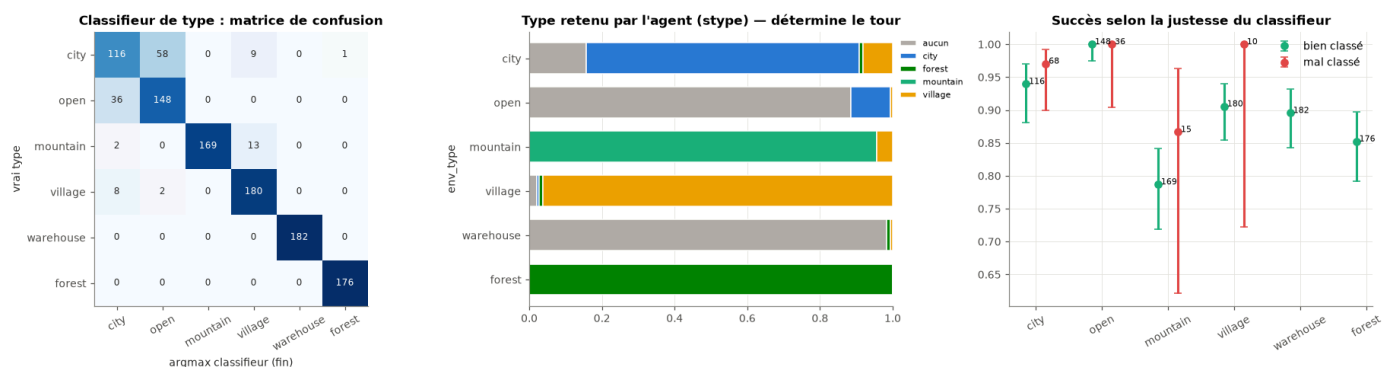
Le prédicat exige simultanément distance horizontale ≤ 2 m, hauteur 2–4 m au-dessus du haut de la boîte englobante, vitesse ≤ 1 m/s, pendant 2 s. Quelle condition bloque ?



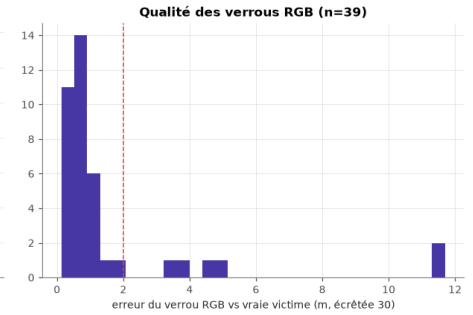
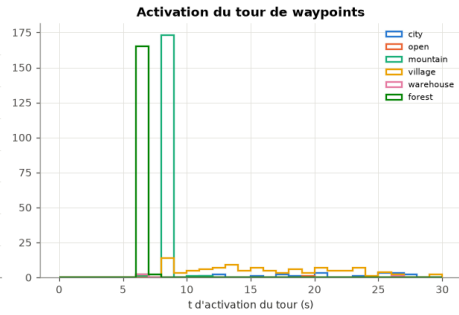
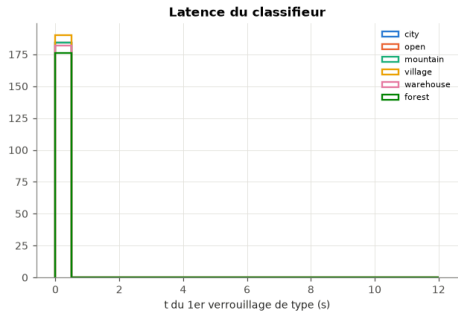
Lecture. Pour les 10 échecs « < 2 m sans prédicat », le drone est **trop haut** dans 95 % des pas passés à moins de 2 m horizontalement (médiane 7.6 m au-dessus de la victime, alors que la fenêtre est de 2–4 m). 5 de ces seeds sont en mountain. Une descente forcée près de la victime suffirait à les récupérer.

8. Internes de l'agent

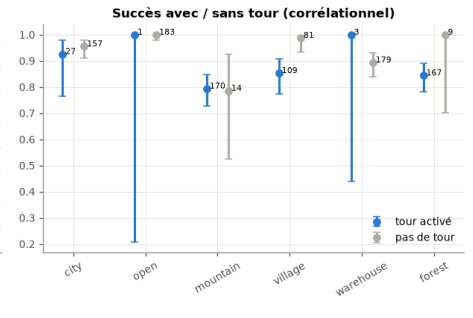
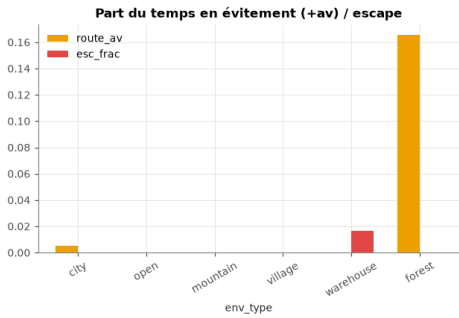
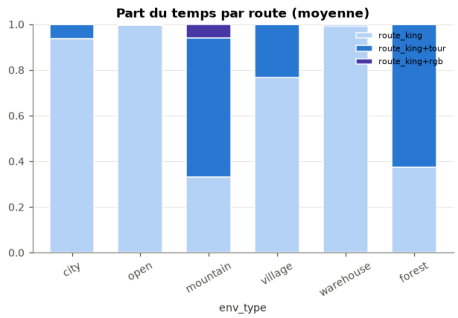
Le champion empile : politique ONNX récurrente (king) + classifieur de type d'environnement (typenet.bin) qui active un **tour** de waypoints (forest/city/mountain/village) + détecteur RGB (victim_rgb_m.onnx , uniquement si montagne confirmée) + **escape** (piège au décollage) + **évitement** (rotation du clue si obstacle < 3 m, forest/city) + route champ (village/warehouse : tout désactivé).



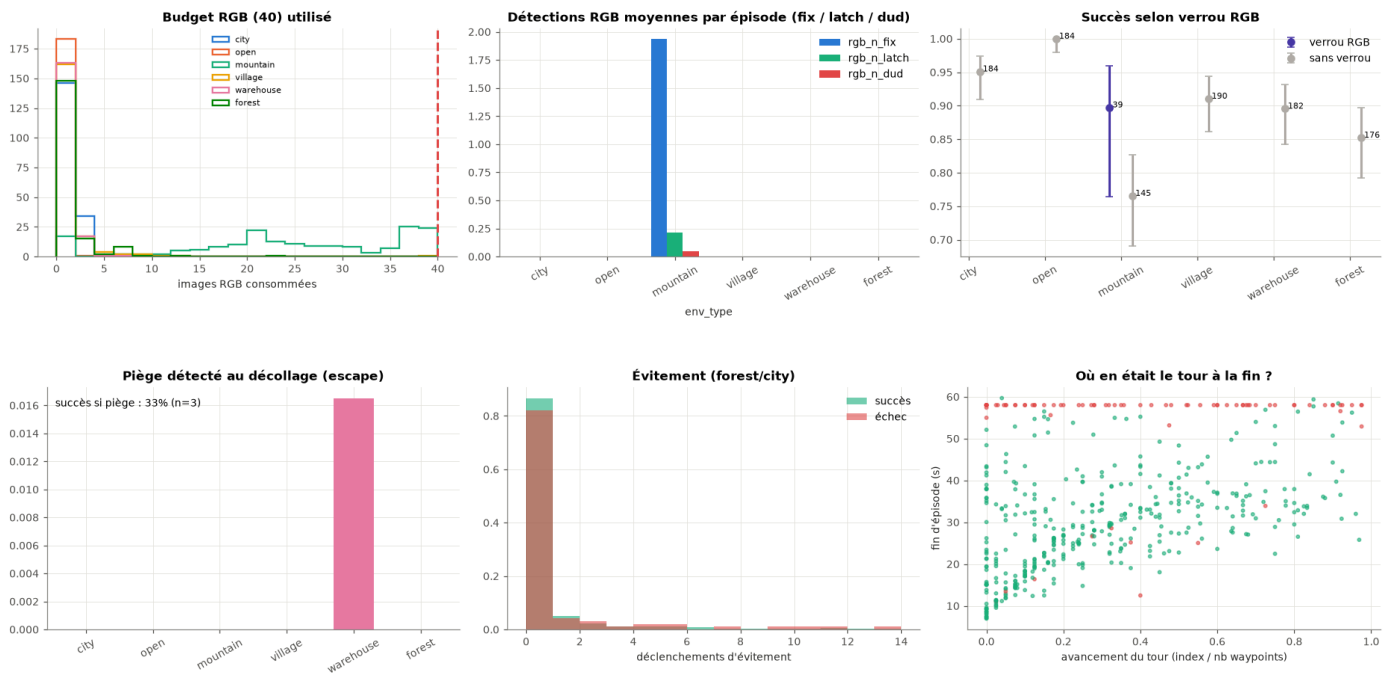
	env_type	city	forest	mountain	open	village	warehouse
kind							
	esc	0	0	0	0	0	3
is_mountain		184	176	360	184	190	182
latched		419	388	380	377	403	367
rgb_latch		0	0	39	0	0	0
rgb_unlatch		0	0	9	0	0	0
tour_on		211	343	365	185	299	185



	route_king	route_king+tour	route_king+rgb	route_av	esc_frac
env_type					
city	0.937	0.063	0.00	0.005	0.000
open	0.998	0.002	0.00	0.000	0.000
mountain	0.330	0.609	0.06	0.000	0.000
village	0.767	0.233	0.00	0.000	0.000
warehouse	0.993	0.007	0.00	0.000	0.016
forest	0.376	0.624	0.00	0.165	0.000



	rgb_used	rgb_n_req	rgb_n_fix	rgb_n_latch	rgb_n_dud
env_type					
city	1.34	0.00	0.00	0.00	0.00
open	1.01	0.00	0.00	0.00	0.00
mountain	24.44	23.73	1.93	0.21	0.05
village	1.56	0.00	0.00	0.00	0.00
warehouse	1.11	0.00	0.00	0.00	0.00
forest	1.52	0.00	0.00	0.00	0.00



Lecture.

- **Classifieur.** Il confond city et open (58/184 seeds city classées open, 36 dans l'autre sens). C'est sans effet : les seeds mal classées réussissent autant ou mieux (97 % et 100 %). Mountain est reconnu à 92 % et warehouse et forest à 100 %.
- **RGB.** Le détecteur n'est utilisé qu'en mountain (24 images sur 40 en moyenne) et ne produit un verrou que dans **39/184 épisodes**. Avec verrou, le succès est de 90 %, contre 77 % sans (corrélational). Augmenter le taux de verrouillage est une piste pour mountain.
- **Escape.** Il ne s'est déclenché que 3 fois (warehouse), sans détecter les pièges du décollage (voir 16.2).

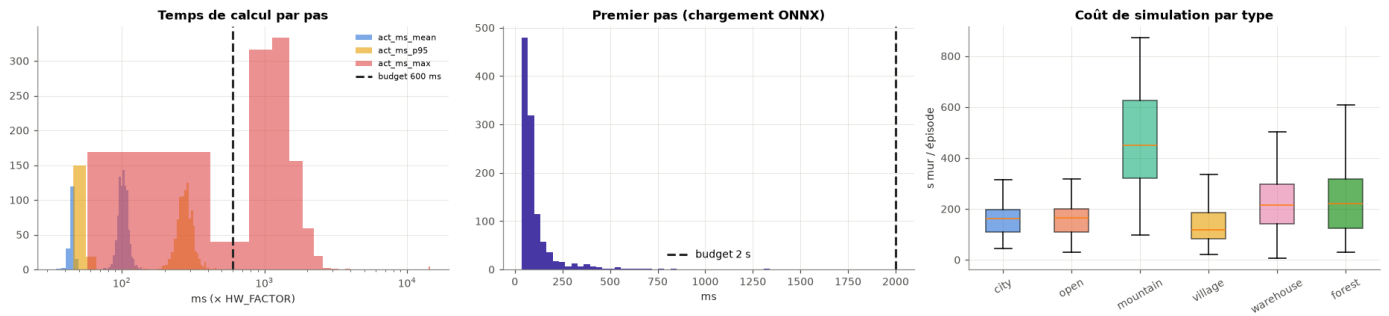
9. Cinématique



10. Budget de calcul

Mesures en process (non calibrées) : `HW_FACTOR` convertit en secondes baseline-équivalentes. Au-delà de 0.6 s/pas (2 s au premier pas), le runner Docker compte un *strike* ; 15 strikes font échouer la seed.

	env_type	city	open	mountain	village	warehouse	forest
act_ms_mean	count	184.0	184.0	184.0	190.0	182.0	176.0
	mean	92.2	99.3	93.1	87.5	94.9	97.8
	std	21.7	24.2	21.4	21.8	22.6	20.7
	min	41.3	41.7	39.3	34.5	43.1	43.2
	25%	92.5	100.3	95.0	87.4	94.5	97.3
	50%	98.6	107.0	100.6	95.3	101.5	103.0
	75%	104.7	112.3	105.6	99.7	106.3	109.3
	max	134.1	130.9	120.6	129.2	156.0	136.8
act_ms_p95	count	184.0	184.0	184.0	190.0	182.0	176.0
	mean	242.8	269.4	246.3	225.1	248.2	260.8
	std	86.3	97.0	85.4	86.2	87.9	83.4
	min	51.8	51.7	50.6	45.9	53.0	51.1
	25%	239.5	270.0	251.9	225.8	241.4	252.6
	50%	264.6	299.8	273.5	254.1	275.2	281.2
	75%	289.0	320.9	293.1	274.2	294.4	305.0
	max	451.8	433.0	379.2	386.1	415.0	459.5
act_ms_max	count	184.0	184.0	184.0	190.0	182.0	176.0
	mean	1185.3	1237.0	1234.5	1030.6	1035.2	1223.4
	std	1140.8	1130.3	1130.3	575.2	528.4	1137.0
	min	62.6	62.8	66.9	57.3	60.1	65.6
	25%	870.3	1002.9	959.0	805.0	844.3	914.5
	50%	1172.3	1204.0	1240.6	1079.4	1086.2	1189.1
	75%	1463.2	1523.7	1501.3	1414.8	1336.4	1469.1
	max	14416.3	14383.3	14447.0	3197.6	2688.0	14383.6
act_ms_first	count	184.0	184.0	184.0	190.0	182.0	176.0
	mean	101.4	100.3	129.9	103.7	103.5	90.1
	std	88.3	83.9	104.6	131.7	94.8	79.4
	min	41.6	35.8	47.0	38.0	38.6	40.3
	25%	56.7	55.3	71.1	53.7	55.0	52.5
	50%	73.8	69.5	88.1	66.1	72.8	67.6
	75%	109.2	101.3	136.6	97.6	103.0	90.3
	max	664.4	622.6	615.5	1338.6	825.8	585.7
wall_sec	count	184.0	184.0	184.0	190.0	182.0	176.0
	mean	161.1	158.5	462.8	147.3	252.3	243.1
	std	61.9	61.7	206.6	89.6	179.9	151.7
	min	44.7	30.4	96.5	22.4	5.7	30.3
	25%	108.3	109.6	319.3	83.3	140.1	124.8
	50%	162.8	165.1	450.5	119.5	213.6	219.8
	75%	198.0	201.4	626.5	184.3	297.2	318.3
	max	349.4	316.6	873.4	391.3	841.6	670.6



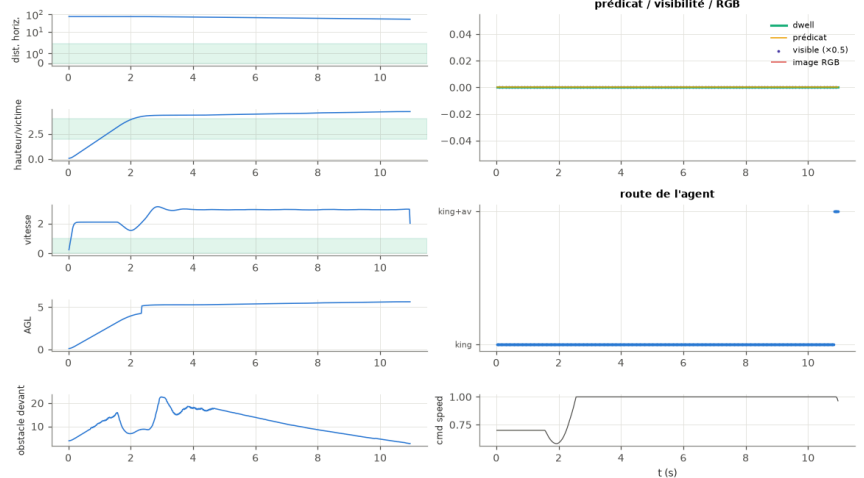
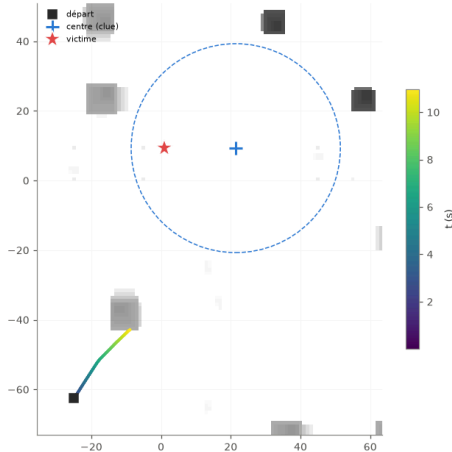
Lecture. En Docker, aucune seed n'échoue pour cause de calcul (100 % d'exécutions propres). En process, on mesure environ 95 ms par pas en moyenne et 250 ms au p95. Les pics maximaux au-delà de 1 s viennent de la contention à 6 workers sur le VPS ; ils ne se retrouvent pas dans les strikes du benchmark. La simulation coûte cher en mountain : 177 s de mur par seed en Docker, soit 4.6 fois le temps simulé.

11. Galerie de trajectoires

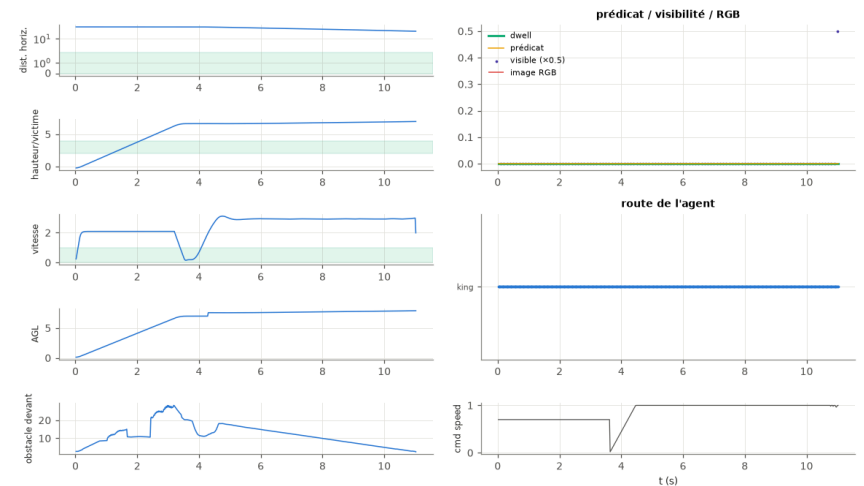
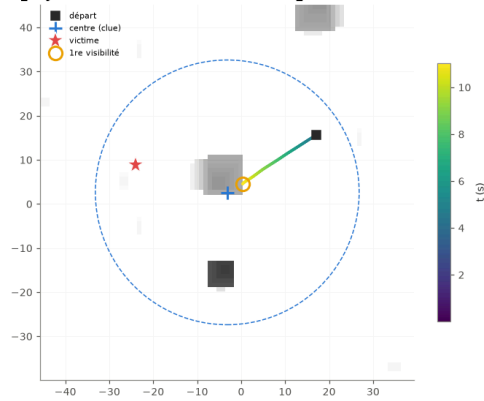
Vue du dessus (carte de hauteur, cercle de recherche, victime, trajet coloré par le temps) + séries temporelles. `plot_episode(group, seed)` pour inspecter n'importe quelle seed.

20 épisodes dans la galerie

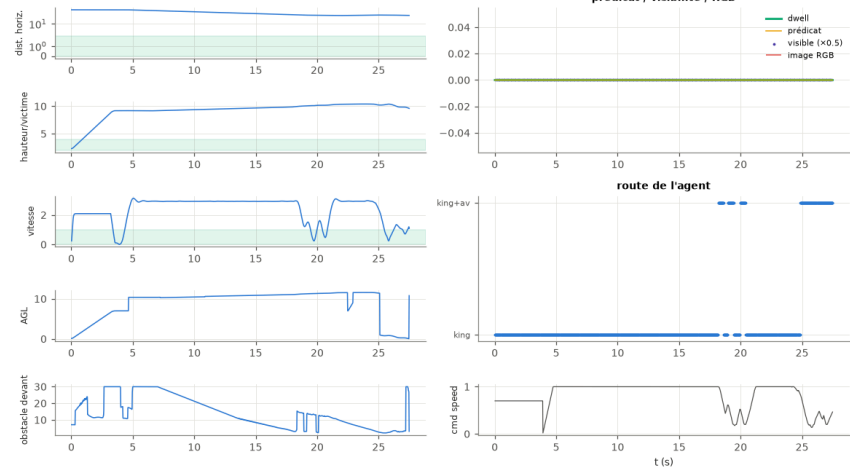
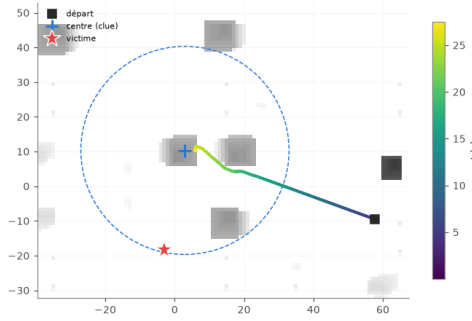
type1_city seed 1000027 — crash (OBSTACLE_COLLISION) score=0.010



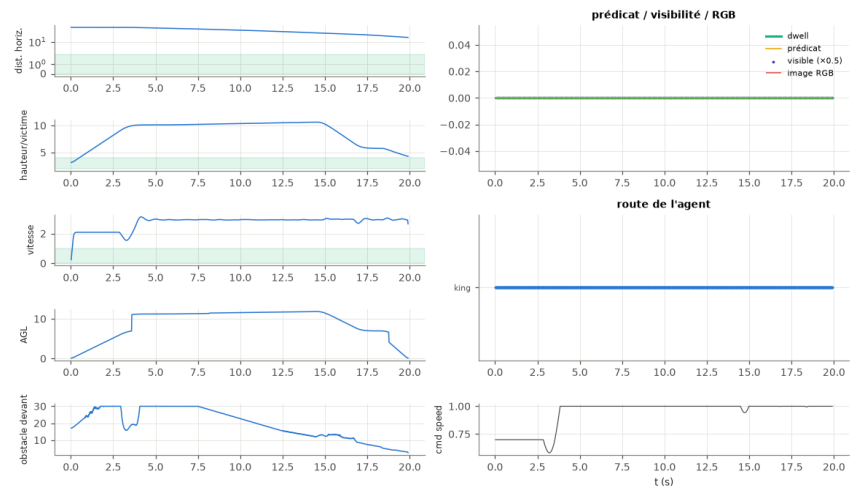
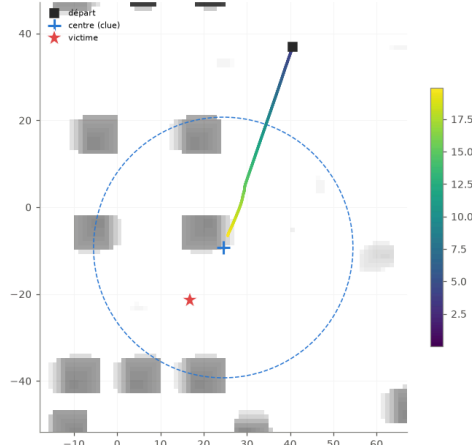
type1_city seed 1000054 — crash (OBSTACLE_COLLISION) score=0.010



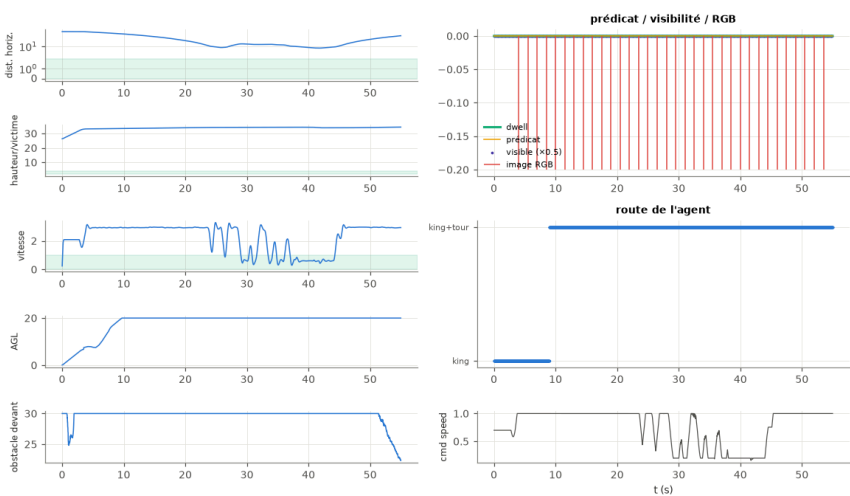
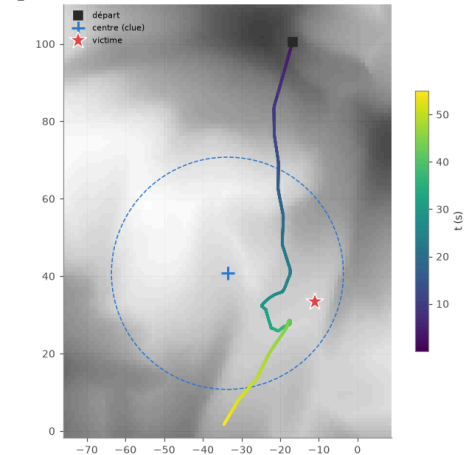
type1_city seed 1000729 — crash (OBSTACLE_COLLISION) score=0.010



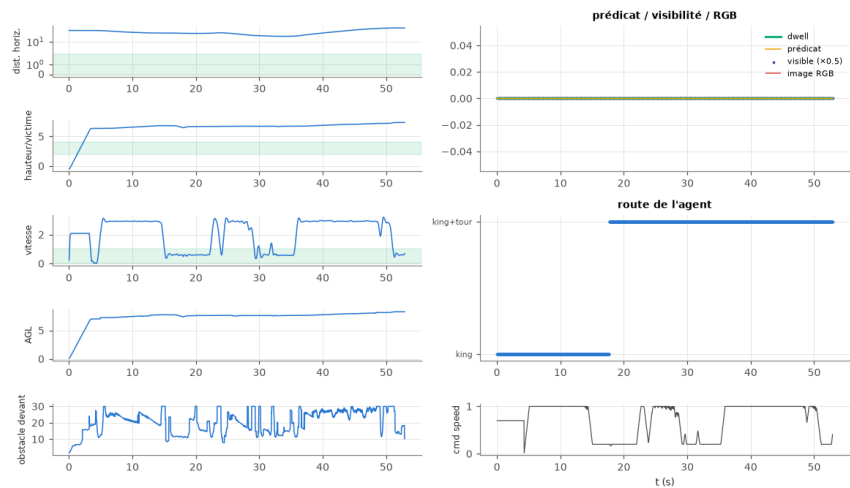
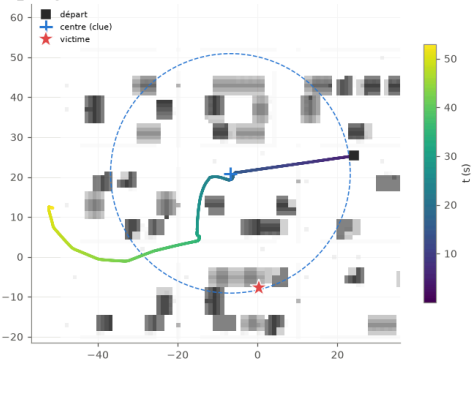
type1_city seed 1000909 — crash (OBSTACLE_COLLISION) score=0.010



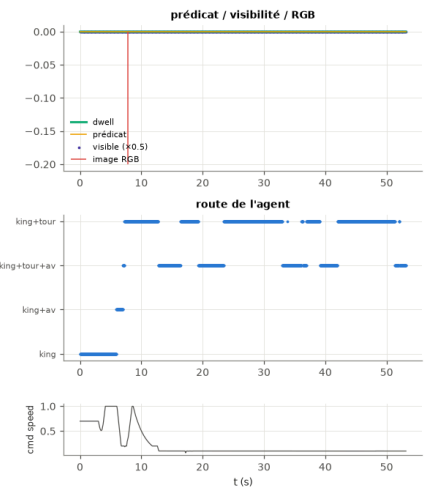
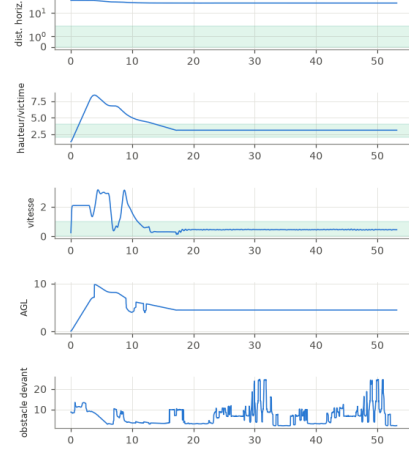
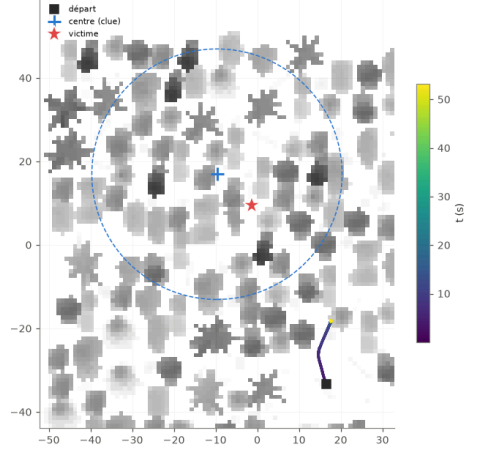
type3_mountain seed 1000255 — infeasible (INFEASIBLE) score=0.010



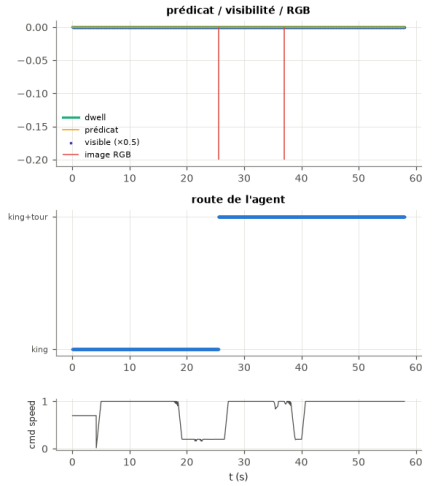
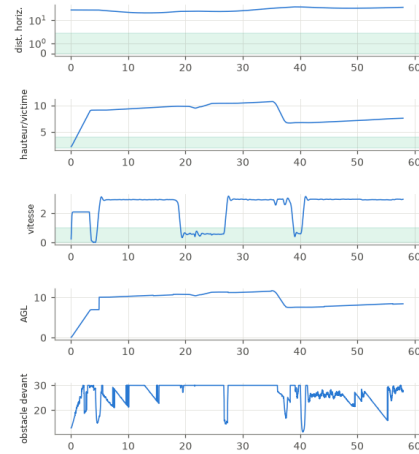
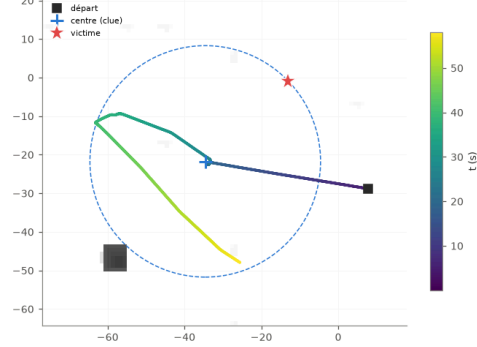
type4_village seed 1000545 — infeasible (INFEASIBLE) score=0.010



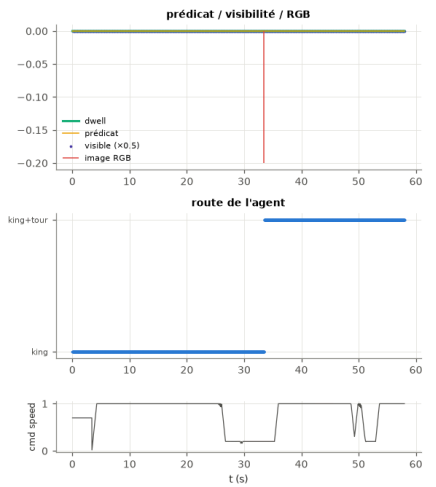
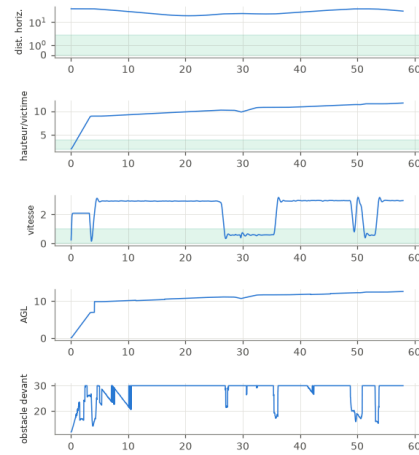
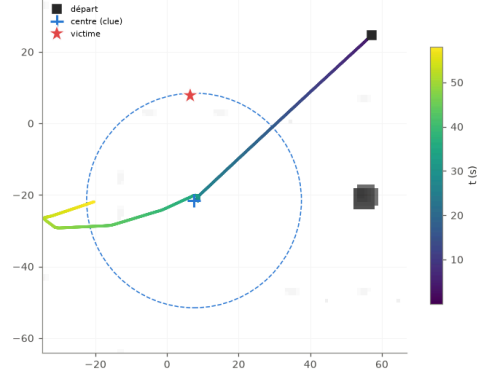
type6_forest seed 1000085 — infeasible (INFEASIBLE) score=0.010



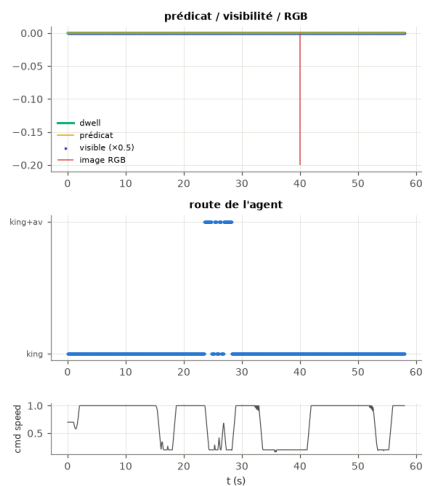
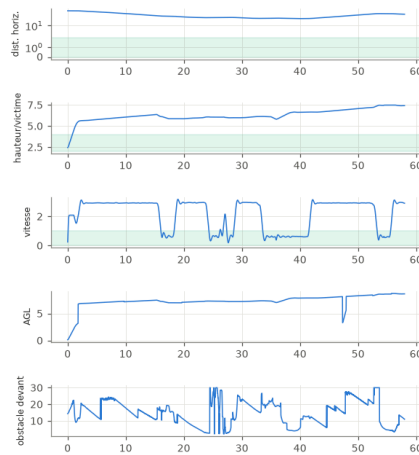
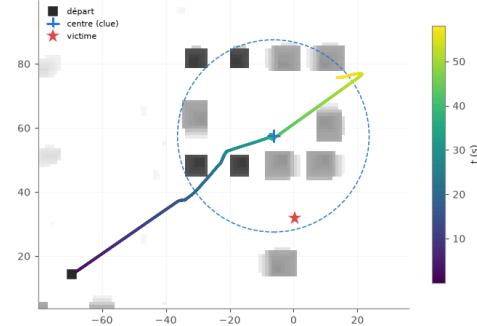
type1_city seed 1000528 — to_never_seen (INFEASIBLE) score=0.010



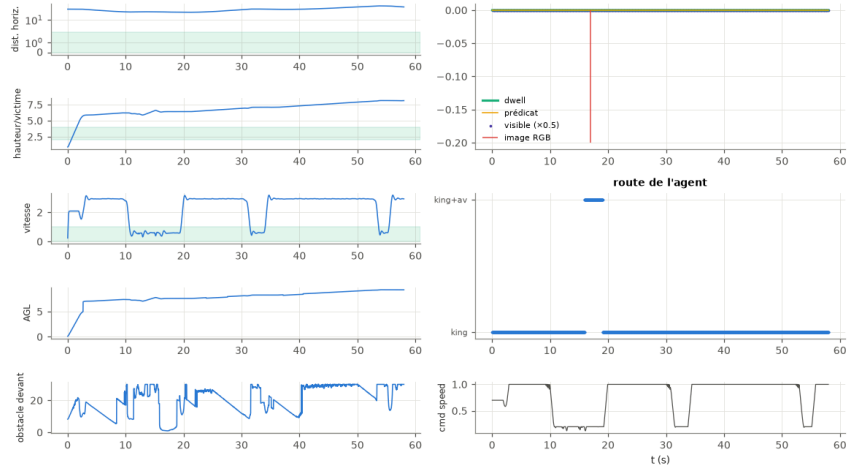
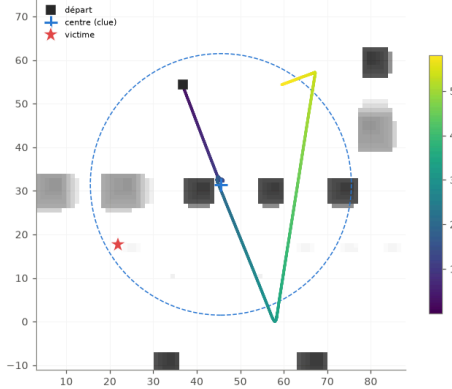
type1_city seed 1000854 — to_never_seen (INFEASIBLE) score=0.010



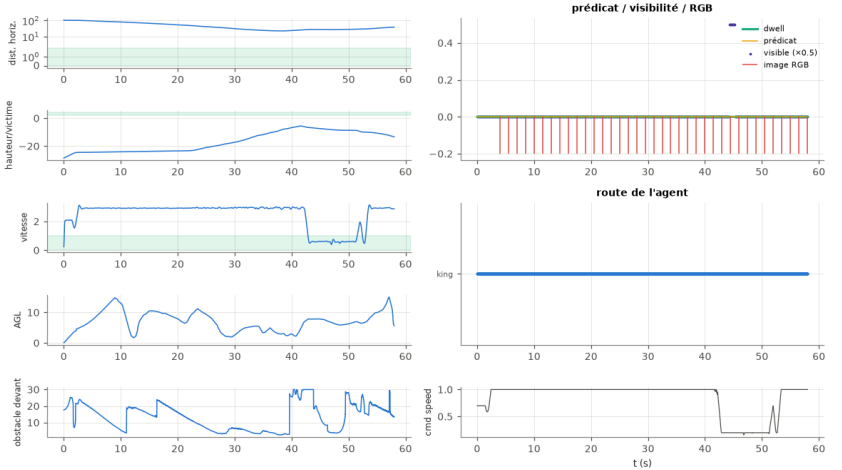
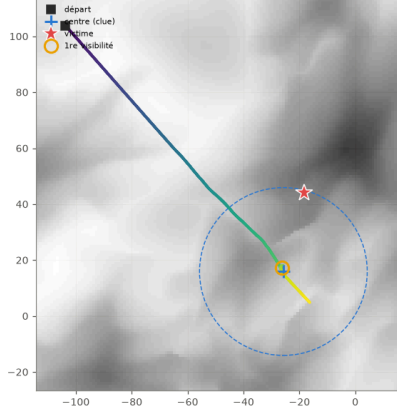
type1_city seed 1000966 — to_never_seen (INFEASIBLE) score=0.010



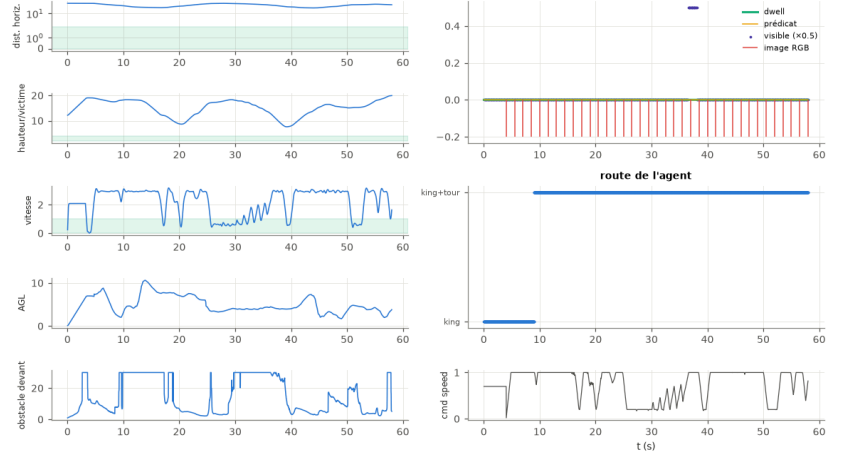
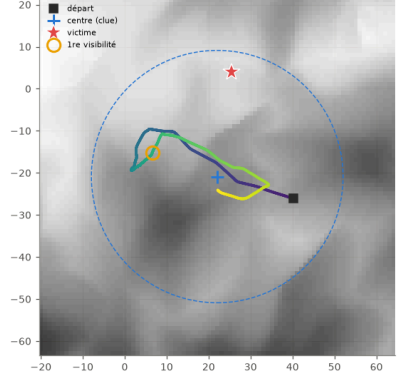
type1_city seed 1001046 — to_never_seen (INFEASIBLE) score=0.010



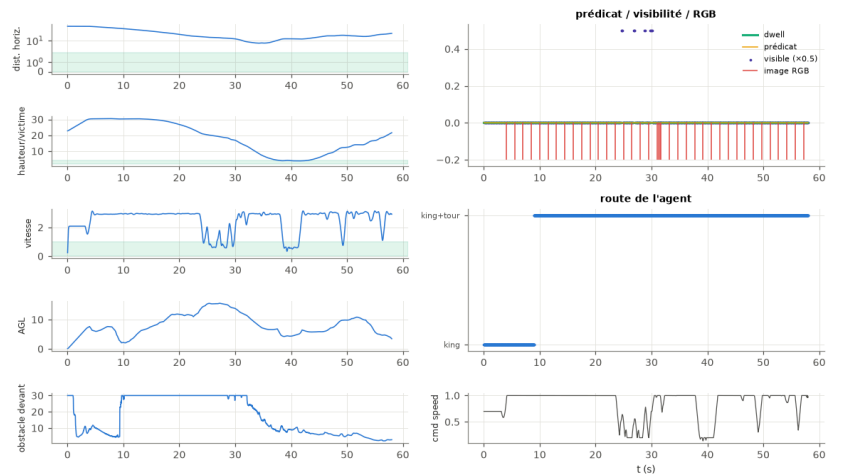
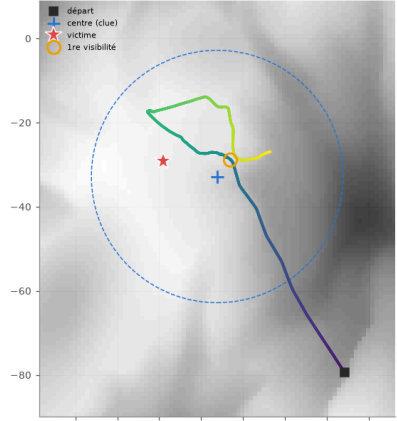
type3_mountain seed 1000083 — to_seen_not_close (INFEASIBLE) score=0.010



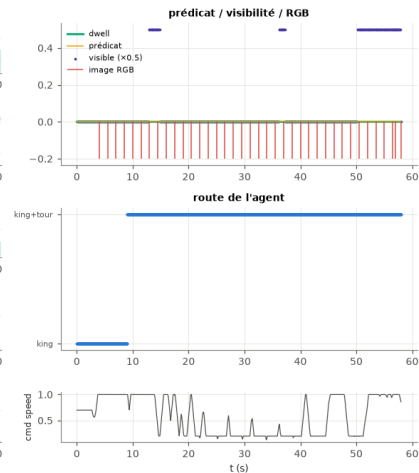
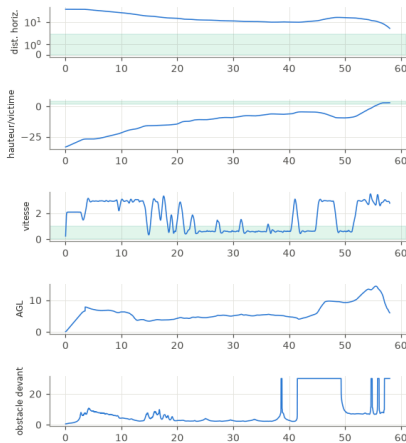
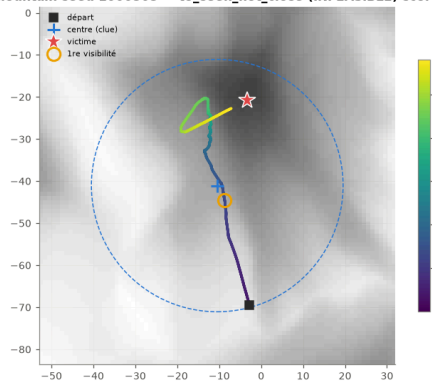
type3_mountain seed 1000221 — to_seen_not_close (INFEASIBLE) score=0.010



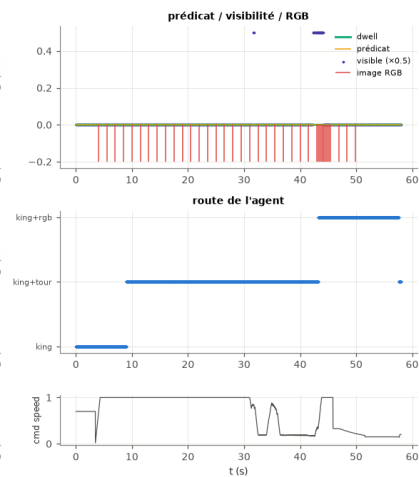
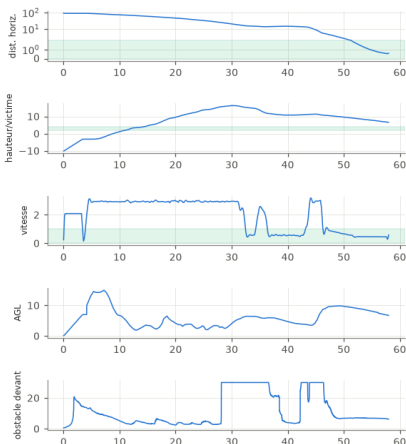
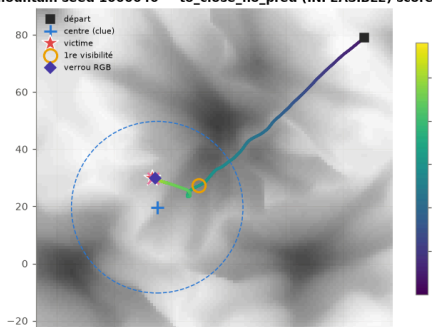
type3_mountain seed 1000226 — to_seen_not_close (INFEASIBLE) score=0.010



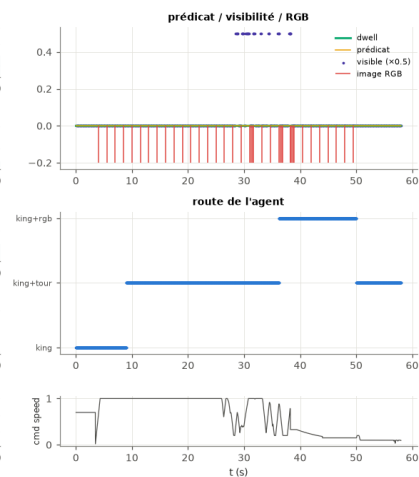
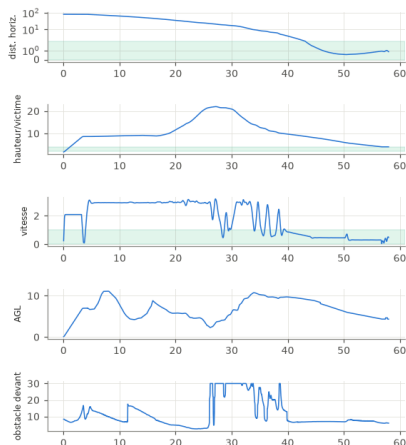
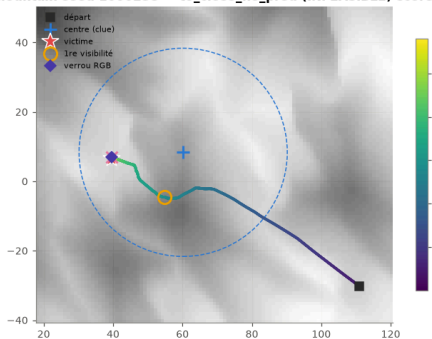
type3_mountain seed 1000303 — to_seen_not_close (INFEASIBLE) score=0.010



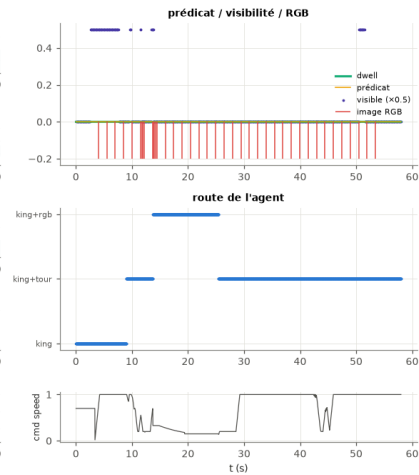
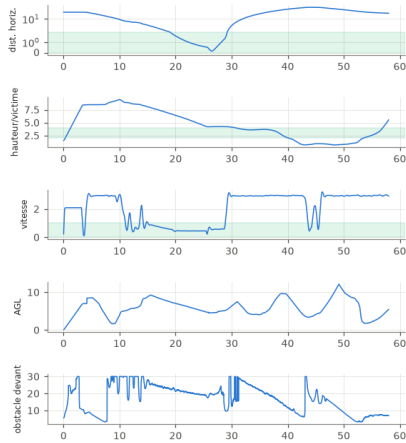
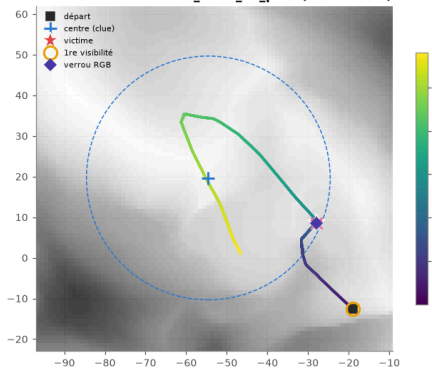
type3_mountain seed 1000040 — to_close_no_pred (INFEASIBLE) score=0.010



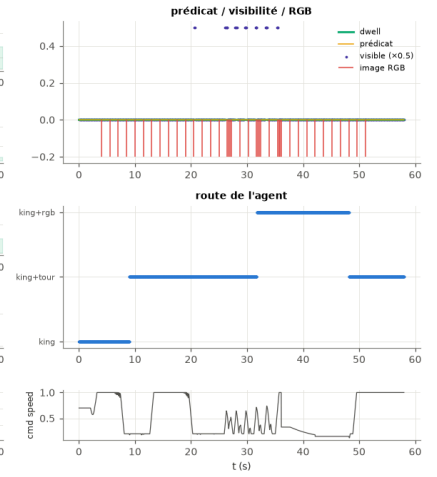
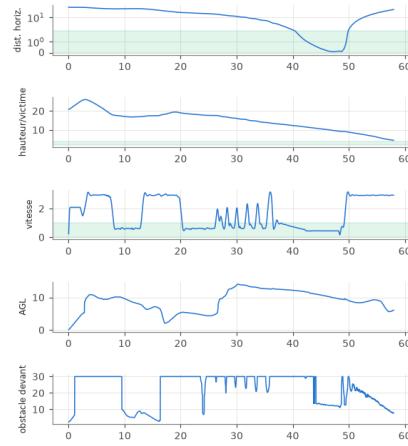
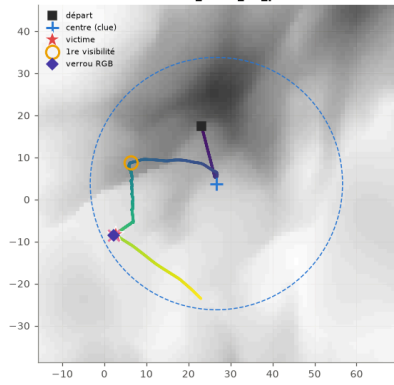
type3_mountain seed 1000238 — to_close_no_pred (INFEASIBLE) score=0.010



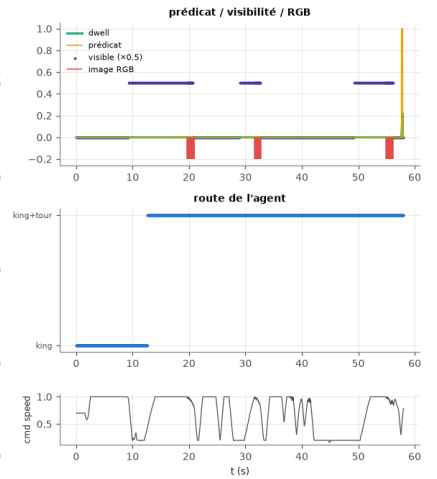
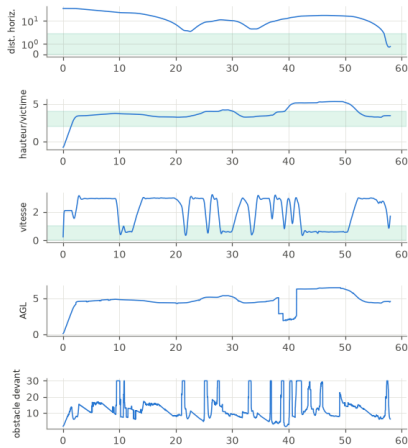
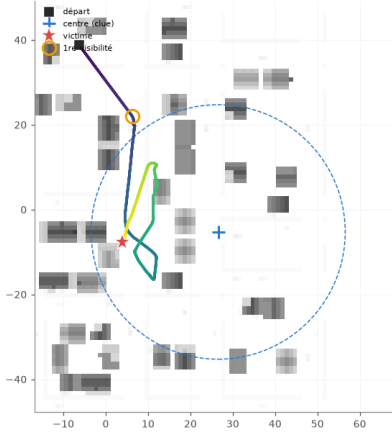
type3_mountain seed 1000245 — to_close_no_pred (INFEASIBLE) score=0.010



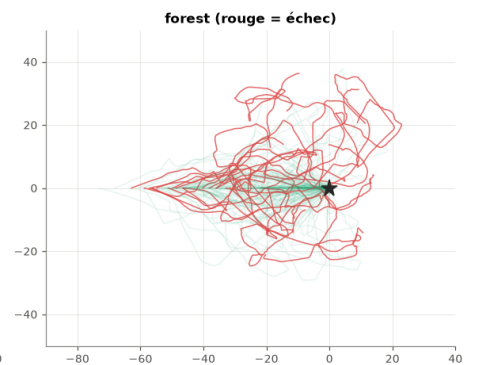
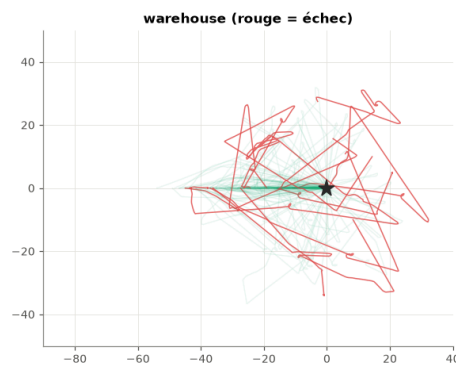
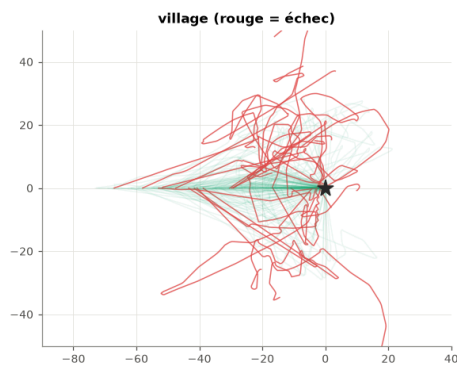
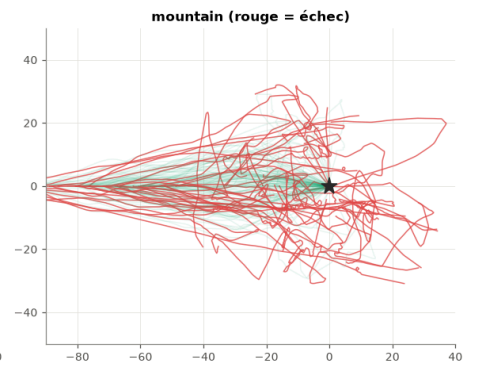
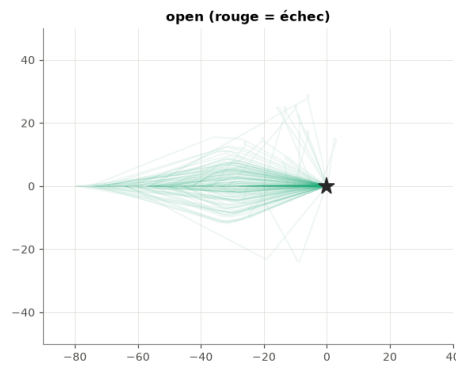
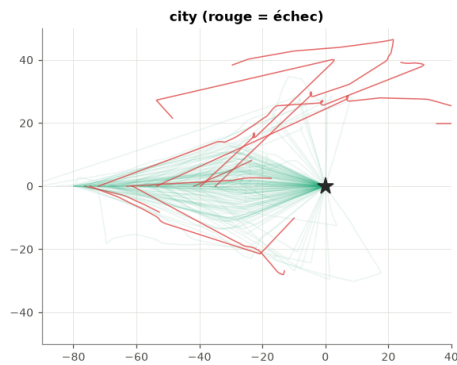
type3_mountain seed 1000616 — to_close_no_pred (INFEASIBLE) score=0.010



type4_village seed 1000514 — to_pred_unstable (INFEASIBLE) score=0.010



Trajets dans le repère victime (départ → victime = axe +x)

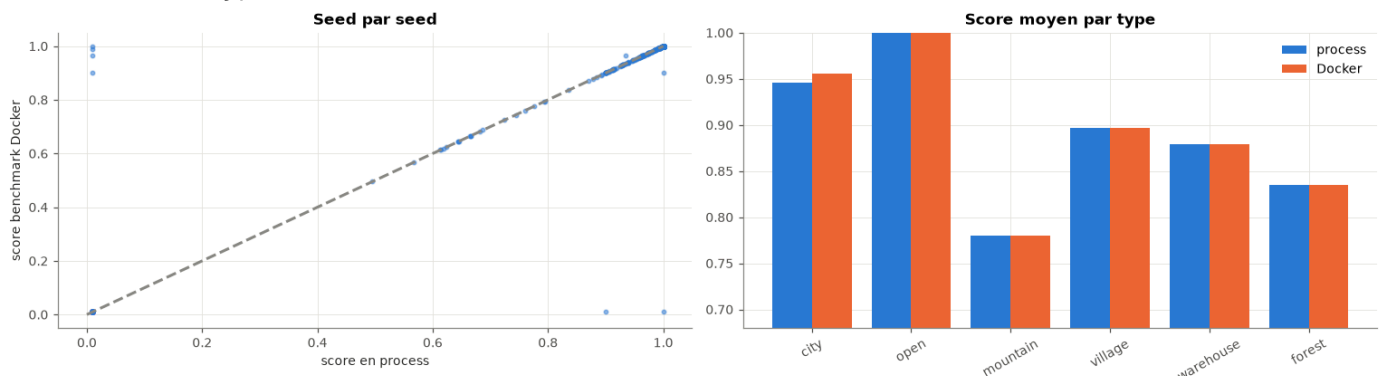


12. Cohérence avec le benchmark Docker (optionnel)

1100 seeds communes ; score en process 0.8900 vs benchmark 0.8917
désaccords succès/échec : 6 (0.5%)

	group	seed	env_type	score_proc	success_proc	score_bench	success_bench
20	type1_city	1000027	city	0.01	False	1.000	True
43	type1_city	1000054	city	0.01	False	0.988	True
404	type1_city	1000309	city	0.90	True	0.010	False
445	type1_city	1000369	city	1.00	True	0.010	False
790	type1_city	1000729	city	0.01	False	0.900	True
938	type1_city	1000909	city	0.01	False	0.966	True

execution_status
seed_done 1100
Name: count, dtype: int64



Lecture. Le process et Docker concordent : 0.8900 contre 0.8917 et 6 désaccords sur 1 100. Les 6 sont en **city** : 4 crashes en process réussissent en Docker, et 2 succès en process échouent en Docker. Tous les autres types sont identiques seed par seed. City est donc légèrement non déterministe (timing ou évitement), et les rollouts en process sont un proxy fiable pour itérer.

13. A/B apparié : champion vs variante

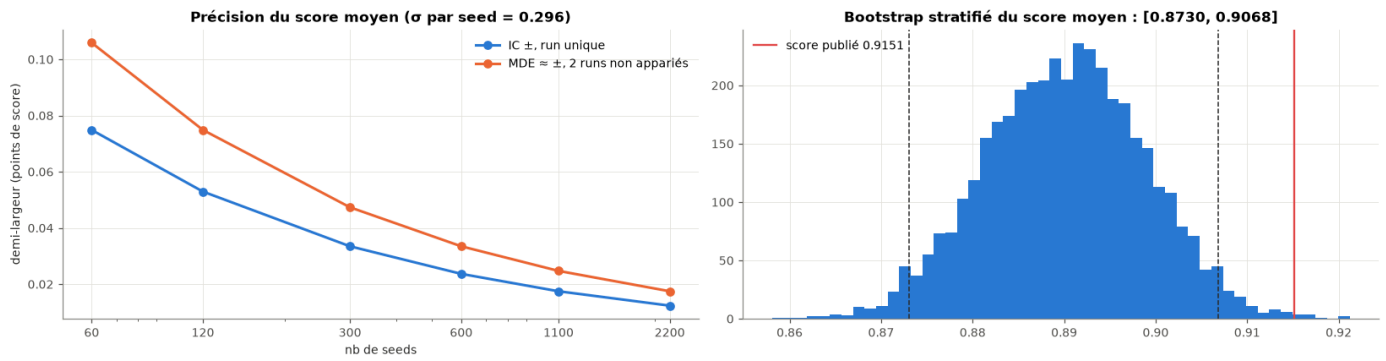
Mêmes seeds des deux côtés → test apparié. Score : IC bootstrap de la différence moyenne et test de permutation (sign-flip). Succès : test exact de McNemar sur les paires discordantes.

RUN_B absent : section ignorée.

14. Puissance statistique

Quelle amélioration peut-on détecter ? Pour un A/B apparié, seul compte le nombre de seeds dont l'issue change. Pour une comparaison non appariée (ex. score public vs set caché), l'écart-type par seed (~0.25) domine.

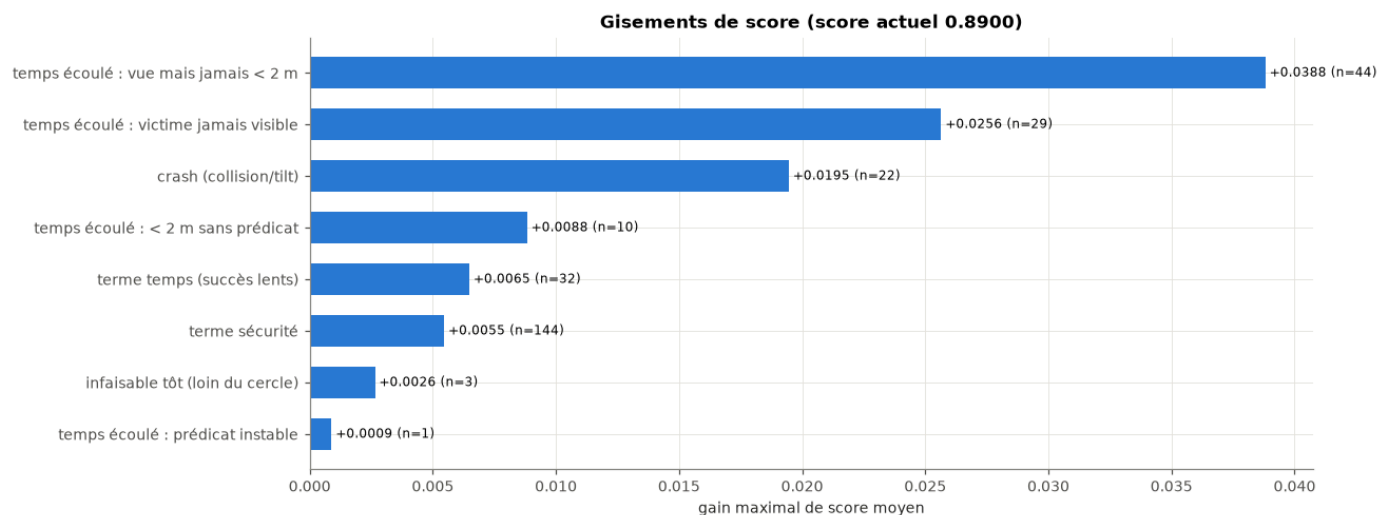
Gain net de seeds réussies nécessaire pour +0.005 de score sur 1 100 seeds : 5.9



15. Synthèse : où chercher le gain ?

Pour chaque classe d'échec : effectif, part de la perte totale, et **gain maximal** si ces seeds étaient réussies avec le score moyen des succès de leur type. Pour le temps : gain si tous les succès confirmaient dans le temps cible. C'est une borne supérieure, pas une prévision.

	poste	n	types	perte_pts	gain_max
0	temps écoulé : vue mais jamais < 2 m	44	mountain:15, forest:13, warehouse:9, village:7	3.960e-02	3.880e-02
1	temps écoulé : victime jamais visible	29	mountain:17, village:5, city:4, warehouse:2, f...	2.610e-02	2.560e-02
2	crash (collision/tilt)	22	forest:9, warehouse:7, city:5, village:1	1.980e-02	1.950e-02
3	temps écoulé : < 2 m sans prédicat	10	mountain:5, village:2, forest:2, warehouse:1	9.000e-03	8.800e-03
4	terme temps (succès lents)	32		6.500e-03	6.500e-03
5	terme sécurité	144		5.500e-03	5.500e-03
6	infaisable tôt (loin du cercle)	3	mountain:1, village:1, forest:1	2.700e-03	2.600e-03
7	temps écoulé : prédicat instable	1	village:1	9.000e-04	9.000e-04

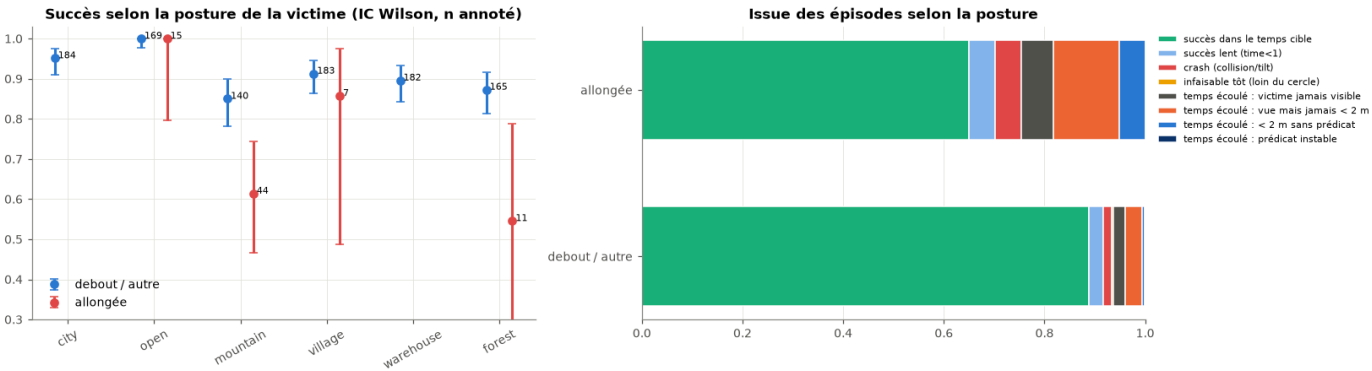


16. Analyses complémentaires (run complet)

Zooms sur les signaux trouvés dans le run 1 100 seeds : posture de la victime, crashes au décollage en warehouse, visibilité sans détection, coût du benchmark officiel et cascade des gains.

16.1 Victimes allongées

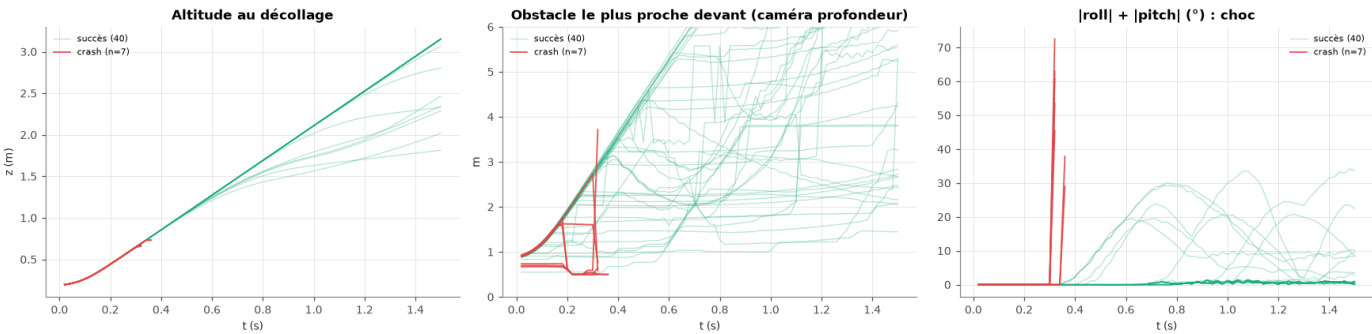
victim_lying	False	True
env_type		
city	0.951	NaN
open	1.000	1.000
mountain	0.850	0.614
village	0.913	0.857
warehouse	0.896	NaN
forest	0.873	0.545



Lecture. Les victimes allongées représentent 7 % des seeds mais 21 % des échecs (23/109). Elles se concentrent en mountain (44) et forest (11). L'échec typique est « vue mais jamais approchée » : la silhouette au sol est petite et peu contrastée. Un apprentissage ou un seuil de détection dédié à ce cas vaut au plus +0.016.

16.2 Warehouse : crashes au décollage

	seed	sim_time	d_start_centre	esc_trap	score
71	1000059	0.32	51.747	False	0.01
209	1000193	0.32	14.987	True	0.01
407	1000397	0.32	54.429	False	0.01
455	1000448	0.36	11.198	True	0.01
887	1000875	0.36	52.916	False	0.01
905	1000900	0.32	25.656	False	0.01
947	1000950	0.32	35.664	False	0.01

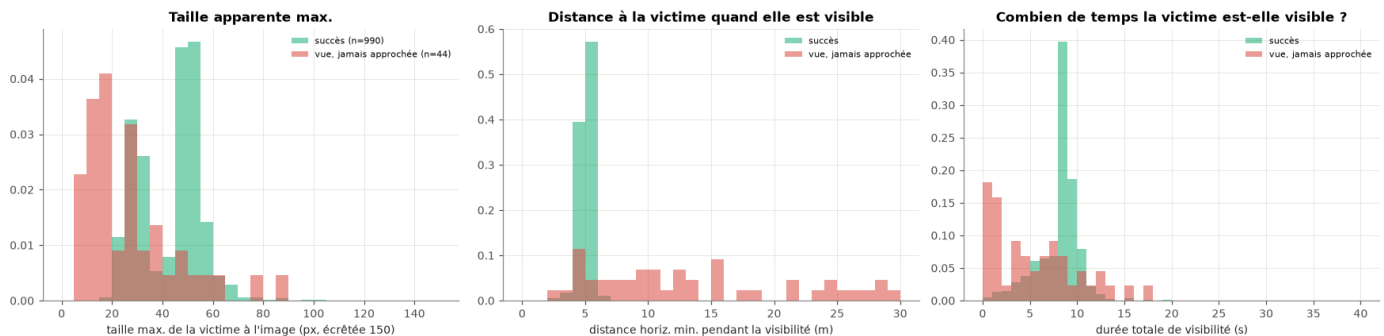


Lecture. Les 7 crashes suivent le même scénario : départ à $z = 0.19$ m, montée verticale pure ($a_{dz} = 1$, environ 2 m/s), choc à $z \approx 0.68$ m au bout de 0.3 s. La caméra voit un obstacle à 0.5–0.75 m devant : le drone démarre sous ou contre une étagère. Le module escape n'a

signalé un piège que dans 2 cas sur 7. **Correctif proposé** : en warehouse, si la profondeur min. au départ est < 1 m, se décaler horizontalement (recul) avant de monter, ou limiter la vitesse de montée tant que la profondeur reste faible. Gain attendu : +0.006, vérifiable de façon déterministe sur ces 7 seeds, sans régression sur les 175 autres seeds warehouse.

16.3 Visible mais pas détectée

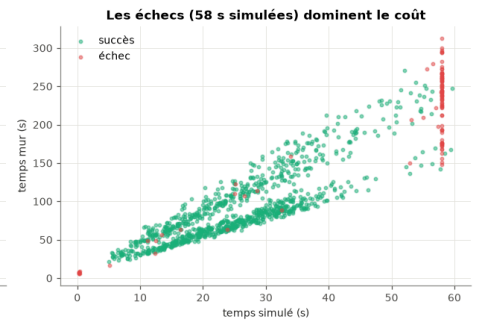
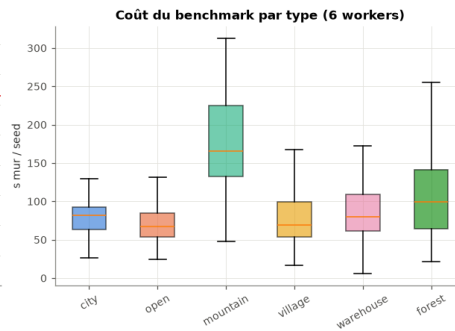
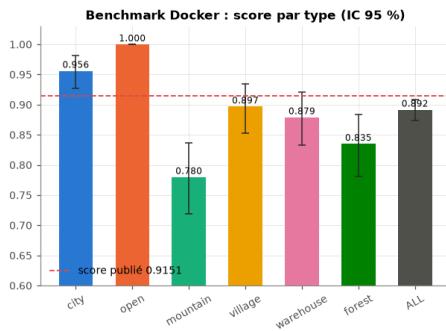
	px_max	hd_min_vis	vis_s
outcome			
crash	20.76	14.40	3.05
success_fast	47.43	5.03	8.60
success_slow	48.34	5.02	9.15
to_close_no_pred	28.23	5.86	3.05
to_pred_unstable	38.97	3.96	25.70
to_seen_not_close	21.00	11.66	4.70



Lecture. Pour les seeds « vue, jamais approchée », la victime reste petite et lointaine, puis sort du champ. Le seuil d'engagement de la politique n'est pas atteint. Deux leviers à tester : (1) une **mémoire de détection** qui retient la position estimée de la victime et y retourne quand le signal disparaît ; (2) un **ralentissement / lacet vers la direction** de la détection faible pour la confirmer. C'est le plus gros gisement (+0.039 au plus).

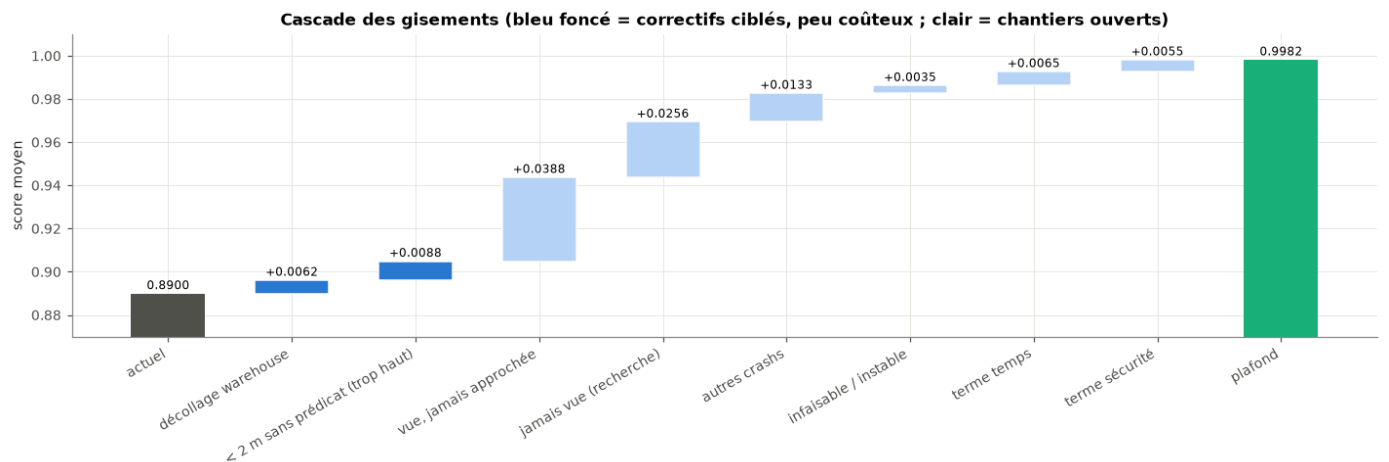
16.4 Benchmark officiel : coût et score par type

	n	score	succes	wall_moy	sim_moy
env_type					
city	184	0.956	0.962	80.471	29.763
open	184	1.000	1.000	68.071	24.144
mountain	184	0.780	0.793	177.566	39.233
village	190	0.897	0.911	80.122	26.202
warehouse	182	0.879	0.896	94.860	21.538
forest	176	0.835	0.852	112.230	26.649



16.5 Cascade des gains (bornes supérieures)

	poste	gain_max
0	décollage warehouse	0.006
1	< 2 m sans prédicat (trop haut)	0.009
2	vue, jamais approchée	0.039
3	jamais vue (recherche)	0.026
4	autres crashes	0.013
5	infaisable / instable	0.004
6	terme temps	0.006
7	terme sécurité	0.005



Lecture. Les deux correctifs ciblés (décollage warehouse, descente près de la victime) rapportent ensemble au plus environ +0.015. Ils sont rapides à valider : seeds identifiées et simulateur déterministe hors city. Le gros du gisement (+0.065) demande de travailler la perception et la recherche (vue sans approche, jamais vue). Il faut l'attaquer avec des A/B appariés sur les 1 100 seeds, ou sur le sous-ensemble mountain/forest pour itérer plus vite.

17. Correctifs : agent_v1 (3 oct. 2026)

analysis/fixes/agent_v1/ = champion + deux correctifs ciblés, désactivables par variable d'environnement (KT_TK0=0 pour le fix 1, KT_TDESC=0 pour le fix 2).

- **Fix 1 (warehouse, décollage).** Au premier pas, si $z \approx 0.191$ m et que le haut de l'image de profondeur (lignes 0:16) voit un obstacle à ≤ 0.6 m, l'agent force l' `_Escape` (désactivé en warehouse dans le champion).

- **Fix 2 (mountain, descente sur verrou RGB).** La descente vers la cible verrouillée était plafonnée à 0.45 m/s, et le minuteur « dud » (6 s) abandonnait la cible pendant la descente. La descente monte maintenant jusqu'à 1.5 m/s quand le drone est à moins de 1.6 m horizontalement, et le minuteur ne compte qu'une fois l'altitude de hover atteinte.

Validation sur de petits lots : les seeds en échec visées, plus des succès témoins du même type.

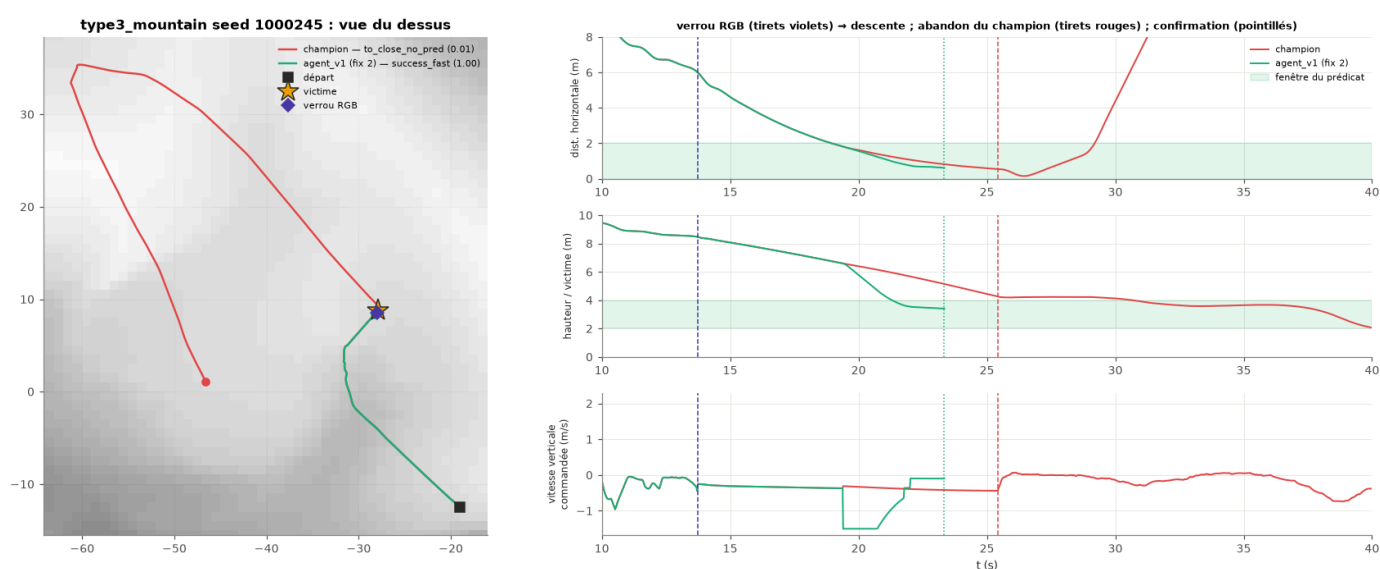
	lot	seed	type	avant	score_avant	après	score_après	gain
0	ciblées	1000245	mountain	to_close_no_pred	0.010	success_fast	1.000	0.990
1	ciblées	1000238	mountain	to_close_no_pred	0.010	success_fast	1.000	0.990
2	ciblées	1000040	mountain	to_close_no_pred	0.010	success_slow	0.976	0.966
3	ciblées	1000616	mountain	to_close_no_pred	0.010	success_slow	0.742	0.732
4	ciblées	1000205	mountain	success_fast	1.000	success_fast	1.000	0.000
5	ciblées	1000490	mountain	success_fast	0.946	success_fast	0.946	0.000
6	ciblées	1000893	mountain	success_fast	0.900	success_fast	0.900	0.000
7	ciblées	1000949	mountain	success_fast	0.961	success_fast	0.961	0.000
8	ciblées	1001063	mountain	success_fast	0.903	success_fast	0.903	0.000
9	ciblées	1000193	warehouse	crash	0.010	success_fast	0.900	0.890
10	ciblées	1000448	warehouse	crash	0.010	success_fast	0.900	0.890
11	ciblées	1000900	warehouse	crash	0.010	success_fast	0.900	0.890
12	ciblées	1000059	warehouse	crash	0.010	crash	0.010	0.000
13	ciblées	1000112	warehouse	success_fast	0.906	success_fast	0.906	0.000
14	ciblées	1000056	warehouse	success_fast	0.997	success_fast	0.997	0.000
15	ciblées	1000397	warehouse	crash	0.010	crash	0.010	0.000
16	ciblées	1000875	warehouse	crash	0.010	crash	0.010	0.000
17	ciblées	1000950	warehouse	crash	0.010	crash	0.010	0.000
18	témoins mountain	1000216	mountain	success_fast	1.000	success_fast	1.000	0.000
19	témoins mountain	1000372	mountain	success_fast	1.000	success_fast	1.000	0.000
20	témoins mountain	1000401	mountain	success_fast	1.000	success_fast	1.000	0.000
21	témoins mountain	1000235	mountain	success_fast	1.000	success_fast	1.000	0.000
22	témoins mountain	1000015	mountain	success_fast	0.947	success_fast	0.947	0.000
23	témoins mountain	1001014	mountain	success_fast	1.000	success_fast	1.000	0.000
24	témoins mountain	1000569	mountain	success_fast	0.994	success_fast	0.994	0.000
25	témoins mountain	1000614	mountain	success_slow	0.568	success_slow	0.568	0.000
26	témoins mountain	1001017	mountain	success_fast	1.000	success_fast	1.000	0.000
27	témoins mountain	1000907	mountain	success_fast	1.000	success_fast	1.000	0.000
28	v2 verrou RGB	1000226	mountain	to_seen_not_close	0.010	success_fast	1.000	0.990
29	v2 verrou RGB	1000363	mountain	to_seen_not_close	0.010	success_fast	1.000	0.990
30	v2 verrou RGB	1000640	mountain	to_seen_not_close	0.010	success_slow	0.893	0.883
31	v2 verrou RGB	1000536	mountain	to_seen_not_close	0.010	success_slow	0.815	0.805
32	v2 verrou RGB	1000614	mountain	success_slow	0.568	success_fast	1.000	0.432
33	v2 verrou RGB	1000017	mountain	success_fast	1.000	success_fast	1.000	0.000
34	v2 verrou RGB	1000061	mountain	success_fast	1.000	success_fast	1.000	0.000
35	v2 verrou RGB	1000005	mountain	success_fast	1.000	success_fast	1.000	0.000
36	v2 verrou RGB	1000015	mountain	success_fast	0.947	success_fast	0.947	0.000
37	v2 verrou RGB	1000216	mountain	success_fast	1.000	success_fast	1.000	0.000
38	v2 verrou RGB	1000083	mountain	to_seen_not_close	0.010	to_seen_not_close	0.010	0.000
39	v2 verrou RGB	1000094	mountain	success_fast	0.999	success_fast	0.999	0.000
40	v2 verrou RGB	1000221	mountain	to_seen_not_close	0.010	to_seen_not_close	0.010	0.000
41	v2 verrou RGB	1000303	mountain	to_seen_not_close	0.010	to_seen_not_close	0.010	0.000
42	v2 verrou RGB	1000372	mountain	success_fast	1.000	success_fast	1.000	0.000
43	v2 verrou RGB	1000310	mountain	to_seen_not_close	0.010	to_seen_not_close	0.010	0.000
44	v2 verrou RGB	1000422	mountain	to_seen_not_close	0.010	to_seen_not_close	0.010	0.000
45	v2 verrou RGB	1000592	mountain	to_seen_not_close	0.010	to_seen_not_close	0.010	0.000

	lot	seed	type	avant	score_avant	après	score_après	gain
46	v2 verrou RGB	1000728	mountain	to_seen_not_close	0.010	to_seen_not_close	0.010	0.000
47	v2 verrou RGB	1000790	mountain	to_seen_not_close	0.010	to_seen_not_close	0.010	0.000

		n	score_avant	score_après	gain_total	récupérées	dégradées	
lot	type							
	ciblées	mountain	9	0.528	0.936	3.678	4	0
		warehouse	9	0.219	0.516	2.670	3	0
témoins	mountain	mountain	10	0.951	0.951	0.000	0	0
v2 verrou	RGB	mountain	20	0.382	0.587	4.100	4	0

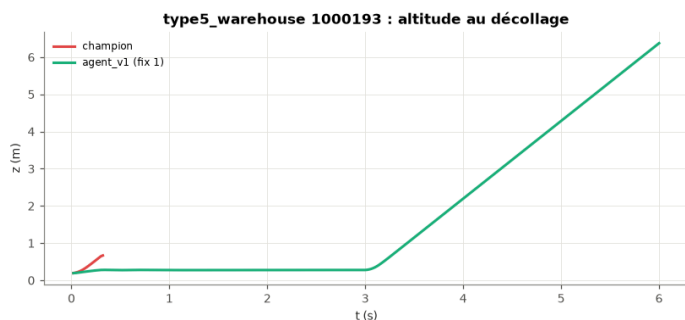
ciblées gain +6.35 pt sur 18 seeds → +0.0058 extrapolé sur 1100 seeds
 témoins mountain gain +0.00 pt sur 10 seeds → +0.0000 extrapolé sur 1100 seeds
 v2 verrou RGB gain +4.10 pt sur 20 seeds → +0.0037 extrapolé sur 1100 seeds

17.1 Mountain 1000245 : avant / après



Lecture. Les deux trajectoires sont identiques jusqu'à $t \approx 19$ s. Le verrou RGB tombe à $t = 13.7$ s (erreur 0.23 m). Le champion descend ensuite à 0.3–0.4 m/s : à $t = 25.4$ s, il est à 0.5 m horizontalement mais encore à 4.2 m au-dessus de la victime, soit 0.2 m au-dessus de la fenêtre du prédictat. Le minuteur « dud » expire à ce moment, la cible est abandonnée et le drone repart chercher ailleurs jusqu'au timeout. Avec le fix 2, la descente passe à 1.5 m/s dès que le drone est au-dessus de la cible : il entre dans la fenêtre 2–4 m à $t \approx 21$ s et confirme à $t = 23.3$ s (score 1.00).

17.2 Warehouse 1000193 : décollage



Lecture. Le champion monte verticalement et percute l'étagère au-dessus de lui en 0.3 s. Avec le fix 1, l'escape le garde au ras du sol pendant 3 s (balayage, puis avance pour sortir de sous l'étagère), puis il monte, reprend la route normale et réussit (0.90).

Bilan. 7 échecs récupérés sur 11 visés (4/4 en mountain, 3/7 en warehouse) et aucun des 17 succès témoins dégradé : +6.35 pts, soit environ **+0.006** extrapolé sur 1 100 seeds. Les 4 crashes warehouse restants sont soit invisibles pour la profondeur (étagère hors champ), soit dos au mur ; le recul et le balayage long ont été testés et sont pires.

17.3 agent_v2 : verrou RGB assoupli

Une sonde (`analysis/fixes/probe_rgb/` , script `analyse_probe_rgb.py`) a enregistré la sortie brute du détecteur RGB sur 443 images de 12 échecs mountain « vue sans approche ». Parmi les 27 détections à $p \geq 0.5$, 24 sont vraies (erreur de position < 3 m). Le filtre `h > 1.0` en rejetait 17, et le plafond de profondeur à 20 m en rejetait 3 ; les 3 acceptées étaient isolées, donc sans verrou. Les 3 fausses ont toutes une profondeur au pixel ≥ 26.8 m.

`agent_v2` : `RP_HMAX` 1.0 $\rightarrow$ 2.0, profondeur max 20 $\rightarrow$ 25 m, verrou dès une détection à $p \geq 0.95$ (`KT_RP_HMAX` , `KT_RP_RANGE` , `KT_RP_ONE`).

Lecture. 4 échecs sur 12 récupérés (1000226 et 1000363 $\rightarrow$ 1.00, 1000640 $\rightarrow$ 0.89, 1000536 $\rightarrow$ 0.81), aucun témoin dégradé et un témoin amélioré (1000614 : 0.57 $\rightarrow$ 1.00) : +4.10 pts sur 20 seeds. Les 4 témoins qui réussissaient sans verrou verrouillent maintenant, avec le même score. Les échecs restants verrouillent trop tard (après 49 s) ou ne détectent rien : c'est la recherche qu'il faut améliorer. À confirmer sur les 184 seeds mountain.

tables écrites dans D:\projets IA\SWARM\sar-task\analysis\diagnostic\tables – figures dans D:\projets IA\SWARM\sar-task\analysis\diagnostic\figures

18. Résultat final : benchmark officiel agent_final vs champion (5 oct. 2026)

`agent_final` (`analysis/agent_final/` , = `agent_v6`) = champion + correctifs fix 1, fix 2, verrou RGB assoupli, plafond d'AGL en mountain et balayage vertical. Les deux agents passent le **même** `swarm benchmark` Docker sur les **mêmes 1 100 seeds**. Comparaison appariée seed par seed, IC 95 % par bootstrap (10 000 tirages).

	n	champion	agent_final	delta	ci_lo	ci_hi	récupérées	perdues
type								
city	184	0.956	0.952	-0.004	-2.210e-02	0.012	1	2
open	184	1.000	1.000	0.000	0.000e+00	0.000	0	0
mountain	184	0.780	0.842	0.062	1.910e-02	0.108	17	4
village	190	0.897	0.897	0.000	0.000e+00	0.000	0	0
warehouse	182	0.879	0.893	0.014	-2.000e-04	0.034	3	0
forest	176	0.835	0.835	0.000	0.000e+00	0.000	0	0
TOTAL	1100	0.892	0.904	0.012	4.000e-03	0.021	21	6

